Kubernetes – Mise en œuvre · Support de cours (INSEE, 14–16 oct. 2026)

Oct 5, 2026 · @Tahar

Mode d'emploi

Ce support accompagne les trois jours de la formation Kubernetes – Mise en œuvre. Il se lit dans l'ordre : 16 modules qui vont du conteneur isolé au cluster sécurisé, avec les commandes et manifestes des 13 travaux pratiques. Vous pouvez le relire après la formation comme référence.

À l'issue de la formation, vous saurez :

1. Expliquer les concepts fondamentaux de Kubernetes et son fonctionnement interne.
2. Installer et configurer un cluster Kubernetes fonctionnel avec kubeadm.
3. Automatiser la gestion des ressources avec des fichiers YAML.
4. Identifier et appliquer les bonnes pratiques d'exploitation : ressources, sécurité, montée en charge, stockage.

Prérequis. Bases de l'administration Linux (shell, paquets, services systemd, réseau IP) et première expérience de Docker ou d'un outil équivalent : lancer un conteneur, lire une image, exposer un port.

Environnement de référence

Élément	Version ou valeur
Kubernetes	1.37 (sortie le 26 août 2026)
Système des nœuds	Ubuntu Server 24.04 LTS, cgroups v2
Runtime de conteneurs	containerd 2.x
Outil d'installation	kubeadm
Réseau des pods (CNI)	Calico 3.31, plage 10.244.0.0/16, encapsulation VXLAN
Plage des Services	10.96.0.0/12 (défaut kubeadm)
Nœuds de chaque stagiaire	`k8s-cp` (control plane), `k8s-w1` et `k8s-w2` (workers)

Élément	Version ou valeur
Application fil rouge	podinfo (interface web et API) + Redis (cache)

Conventions

- Dans les blocs de commandes, `[cp]` signifie « sur le control plane », `[tous]` « sur les trois nœuds », `[w1]` « sur le premier worker ». Sans précision, la commande se lance sur `k8s-cp`, où `kubectl` est configuré.
- Les lignes commençant par `#` sont des commentaires ; ce qui suit `#` en fin de ligne décrit le résultat attendu.
- Les fichiers YAML sont indentés avec 2 espaces. Les noms de fichiers (`30-podinfo.yaml`) sont ceux des TP.
- Chaque module se termine par un encadré **À retenir** et quelques **questions de révision**.

Des encadrés **TP — à faire maintenant** jalonnent les modules : ils indiquent le moment de chaque TP (jour, heure, durée), les sections qu'il met en pratique et son point de contrôle. Les consignes pas à pas, les fichiers et les corrigés des TP sont sur GitHub : github.com/tahero-madi/kubernetes-tp.

Plan des trois jours

Jour	Modules	Travaux pratiques
J1 — mercredi 14 octobre : comprendre et installer	1. Des conteneurs à l'orchestration · 2. Architecture · 3. Écosystème Cloud · 4. Installer avec kubeadm · 5. Gérer les nœuds	TP1 Préparer les nœuds et premiers conteneurs (ctr) · TP2 Initialiser le cluster · TP3 Gérer les nœuds
J2 — jeudi 15 octobre : décrire et exploiter	6. YAML et manifestes · 7. Pods, Deployments, Services · 8. ConfigMaps et Secrets · 9. Cycle de vie · 10. Surveiller et diagnostiquer	TP4 Premiers manifestes · TP5 Application fil rouge · TP6 Mises à jour et probes · TP7 Chasse aux pannes
J3 — vendredi 16 octobre : encadrer, sécuriser, faire grandir	11. Ressources · 12. Labels et stratégies de déploiement · 13. Opérateurs et Helm · 14. Autoscaling · 15. Sécurité · 16. Volumes · Atelier de synthèse	TP8 Quotas et labels · TP9 Helm et cert-manager · TP10 HPA · TP11 Sécurité · TP12 Persistance · TP13 Atelier de migration

Module 1 — Des conteneurs à l'orchestration

Un conteneur isole un processus sur un hôte ; Kubernetes pilote des centaines de conteneurs répartis sur plusieurs machines en ramenant sans cesse l'état réel vers l'état que vous avez décrit. Ce module pose les bases nécessaires pour comprendre pourquoi un orchestrateur est devenu indispensable.

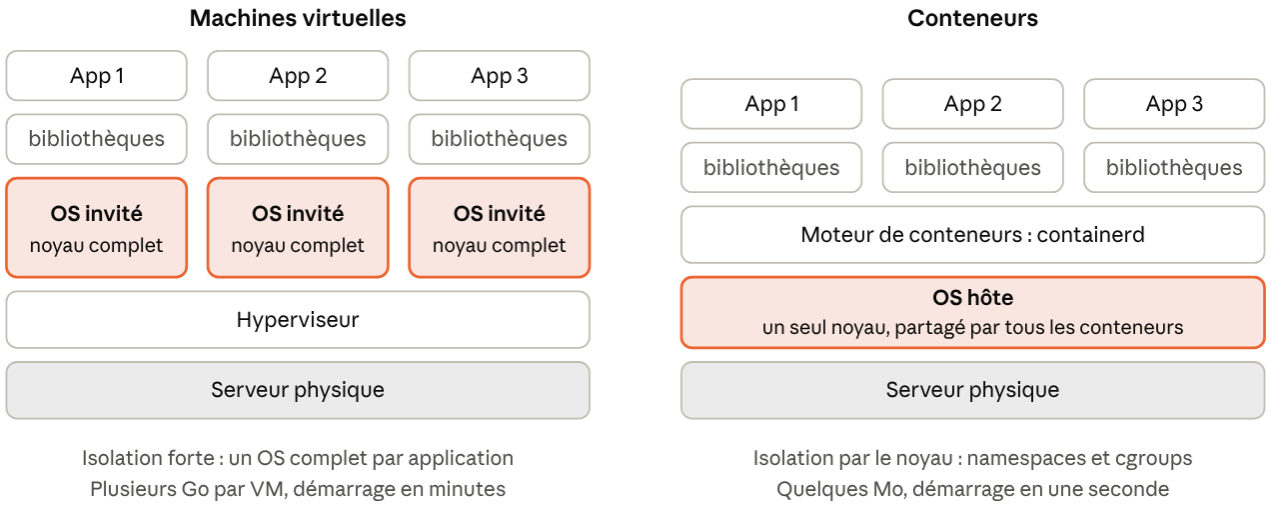
1.1 Du serveur physique au conteneur

Trois générations d'hébergement se sont succédé :

1. **Le serveur physique.** Une application par machine, pour éviter les conflits de bibliothèques. Les serveurs sont sous-utilisés et en commander un prend des semaines.
2. **La machine virtuelle (VM).** Un hyperviseur découpe le serveur en machines complètes, chacune avec son propre noyau. L'isolation est forte, mais chaque VM embarque un système entier : des gigaoctets et un démarrage en minutes.
3. **Le conteneur.** Les applications partagent le noyau de l'hôte ; seul l'espace utilisateur (bibliothèques, binaires) est propre à chacune. Un conteneur pèse quelques dizaines de mégaoctets et démarre en une seconde.

Chaque VM embarque un OS complet ; les conteneurs partagent le noyau de l'hôte

Trois applications hébergées de deux façons sur le même serveur



VM et conteneurs · trois applications, deux façons de les héberger

À gauche, chaque application emporte un système complet ; à droite, un seul noyau sert toutes les applications, d'où la légèreté des conteneurs.

Critère	Machine virtuelle	Conteneur
Ce qui est isolé	Une machine complète, matériel virtuel compris	Un ou plusieurs processus
Noyau	Un par VM	Partagé avec l'hôte
Taille typique	Plusieurs Go	Quelques dizaines à centaines de Mo
Démarrage	Dizaines de secondes à minutes	Moins d'une seconde à quelques secondes
Niveau d'isolation	Fort (frontière matérielle)	Bon, mais le noyau est commun
Usage typique	Isoler des systèmes, des clients, des OS différents	Empaqueter et déployer des applications

Les deux se combinent : dans cette formation, vos conteneurs tournent dans des VM Cloud. C'est le cas de la plupart des clusters Kubernetes en production.

1.2 Un conteneur n'est qu'un processus

Il n'existe pas d'objet « conteneur » dans le noyau Linux. Un conteneur est un processus ordinaire auquel on applique trois mécanismes.

Les namespaces donnent au processus sa propre vue du système :

Namespace	Ce qu'il isole
`pid`	La liste des processus : le conteneur voit son processus principal comme PID 1
`net`	Interfaces réseau, adresses IP, tables de routage, ports
`mnt`	Les points de montage, donc l'arborescence de fichiers
`uts`	Le nom d'hôte
`ipc`	Files de messages et mémoire partagée
`user`	Les identifiants d'utilisateurs (root dans le conteneur, non privilégié dehors)
`cgroup`	La vue de l'arborescence des cgroups

Les cgroups (control groups) limitent et mesurent ce que consomme un groupe de processus : CPU, mémoire, entrées-sorties, nombre de processus. C'est sur eux que reposent les `requests` et `limits` de Kubernetes (module 11). Les distributions récentes, dont Ubuntu 24.04, utilisent les cgroups v2, une hiérarchie unifiée. Kubernetes a déprécié les cgroups v1 en version 1.35 : par défaut, le kubelet refuse désormais de démarrer sur un nœud en cgroups v1.

Le système de fichiers en couches (overlayfs) empile les couches en lecture seule de l'image et ajoute une couche inscriptible propre au conteneur. Plusieurs conteneurs issus d'une même image partagent donc ses couches sur le disque. Ce qu'un conteneur écrit disparaît avec lui, d'où les volumes (module 16).

Vous pouvez le vérifier sur n'importe quel Linux, sans Docker :

```
sudo unshare --pid --fork --mount-proc bash   # nouveau namespace pid
ps aux                                        # seuls bash et ps
apparaissent, bash a le PID 1
exit
```

1.3 Images et registres

Une **image** est un paquet en lecture seule : des couches de fichiers et une configuration (commande de démarrage, variables d'environnement, utilisateur, ports). Son format est standardisé par l'OCI (Open Container Initiative) depuis 2015 : une image construite avec Docker fonctionne avec containerd, CRI-O ou Podman.

Une image est désignée par une référence en quatre parties :

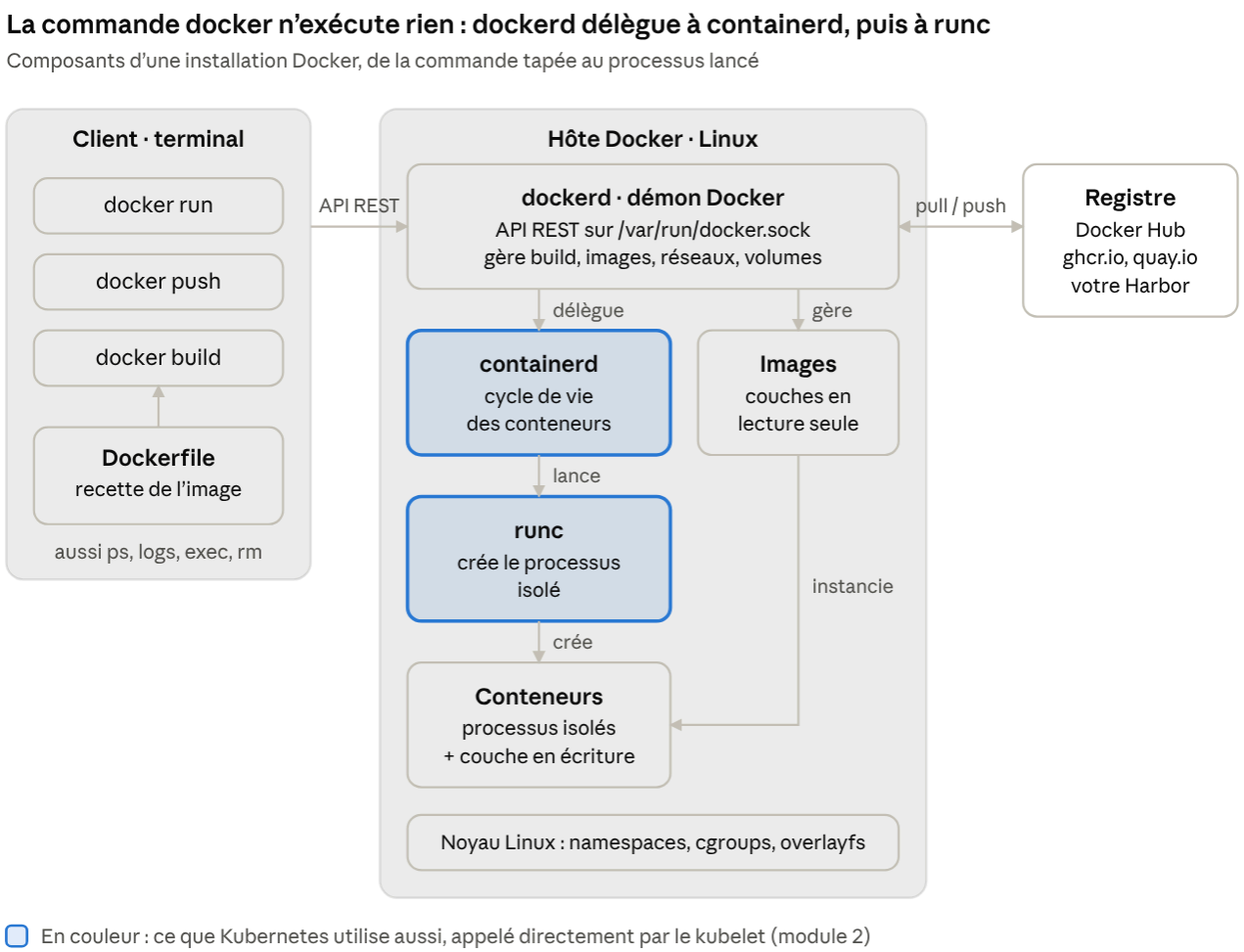
```
ghcr.io / stefanprodan / podinfo : 6.7.0 @sha256:...
registre   espace        nom      tag     empreinte (digest)
```

- Sans registre, c'est Docker Hub : `nginx` signifie `docker.io/library/nginx:latest`.
- Un **tag** est une étiquette déplaçable : `6.7.0` peut être republié. Une **empreinte** (`@sha256:...`) désigne un contenu exact et immuable.
- Évitez `latest` en production : vous ne savez plus quelle version tourne, et deux nœuds peuvent exécuter des versions différentes.

Les **registres** stockent et distribuent les images : Docker Hub, GitHub Container Registry (`ghcr.io`), Quay (`quay.io`), le registre officiel de Kubernetes (`registry.k8s.io`), ou un registre privé (Harbor, GitLab). Docker Hub limite le nombre de téléchargements anonymes par adresse IP ; c'est pourquoi les TP utilisent surtout d'autres registres.

1.4 Docker, containerd, runc : qui fait quoi

« Docker » désigne un ensemble d'outils empilés. Kubernetes n'utilise que la partie basse.



architecture Docker · de la commande au processus

Le client et dockerd n'existent que sur un poste Docker. containerd et runc, en couleur, sont aussi ceux qu'emploie Kubernetes sur chaque nœud : le kubelet les appelle sans passer par dockerd.

Composant	Rôle	Utilisé par Kubernetes ?
`docker` (CLI) et `dockerd`	Interface pour les humains : construire, lancer, publier	Non
containerd	Runtime de haut niveau : télécharge les images, gère le cycle de vie des conteneurs	Oui, via l'interface CRI
runc	Runtime de bas niveau : crée réellement le processus avec namespaces et cgroups	Oui, appelé par containerd

Depuis la version 1.24, Kubernetes ne parle plus directement au moteur Docker : il dialogue avec un runtime compatible CRI (Container Runtime Interface), comme containerd ou CRI-O. Vos images Docker fonctionnent sans changement, puisqu'elles respectent le format OCI. Kubernetes 1.35 a été la dernière version à prendre en charge containerd 1.x : depuis la 1.36, il faut containerd 2.0 ou plus.

1.5 Les limites d'un hôte unique

Imaginez une application de 10 conteneurs répartis sur 5 serveurs, avec Docker seul. Il faut résoudre à la main toutes ces questions :

Besoin	Sans orchestrateur	Avec Kubernetes
Placement	Choisir à la main le serveur de chaque conteneur	Le scheduler choisit selon les ressources libres et vos contraintes
Panne d'un conteneur	Script de surveillance, `restart: always` local	Redémarrage ou remplacement automatique
Panne d'un serveur	Relancer ailleurs à la main	Les pods sont recréés sur les nœuds restants
Montée en charge	Lancer des copies et reconfigurer le répartiteur	`kubectl scale` ou autoscaling (module 14)
Mise à jour sans coupure	Procédure manuelle conteneur par conteneur	Rolling update et retour arrière (module 9)

Besoin	Sans orchestrateur	Avec Kubernetes
Trouver un service	Adresses IP en dur, fichiers de configuration	Nom DNS stable et répartition de charge (module 7)
Configuration et secrets	Fichiers copiés sur chaque serveur	ConfigMaps et Secrets (module 8)
Stockage	Monter les disques à la main	Volumes demandés par déclaration (module 16)

Docker Compose décrit bien une application, mais sur une seule machine. Il ne répond pas aux questions de plusieurs hôtes, de panne et de mise à l'échelle.

1.6 Kubernetes en bref

Kubernetes est une plateforme open source qui automatise le déploiement, la mise à l'échelle et l'exploitation d'applications conteneurisées sur un ensemble de machines appelé **cluster**.

- **Le nom** vient du grec *kubernḗtēs*, le timonier. L'abréviation K8s remplace les 8 lettres centrales par leur nombre.
- **L'origine** : Google exploitait depuis le milieu des années 2000 un orchestrateur interne, Borg. Kubernetes en reprend les idées ; il est annoncé en 2014, sort en version 1.0 en juillet 2015 et devient le premier projet de la CNCF (Cloud Native Computing Foundation), sous l'égide de la Linux Foundation.
- **La gouvernance** : un projet communautaire organisé en groupes d'intérêt (SIG) ; aucun éditeur ne le contrôle seul.
- **Le rythme des versions** : trois versions mineures par an, numérotées `1.x.y`. Chaque version mineure est maintenue environ 14 mois. On met à jour un cluster une version mineure à la fois (1.36 vers 1.37, jamais 1.35 vers 1.37).

Ce que Kubernetes apporte : placement automatique, auto-réparation, mise à l'échelle horizontale, déploiements progressifs et retours arrière, découverte de services et répartition de charge, gestion de la configuration et des secrets, orchestration du stockage, traitements par lots, et une API extensible.

Ce que Kubernetes ne fait pas : il ne construit pas vos images, ne fournit pas d'intégration continue, ni de collecte de logs ou de supervision clés en main. Ces briques s'ajoutent autour de lui.

Un exemple proche de vous. La plateforme de datascience Onyxia, développée à l'INSEE et utilisée pour le SSP Cloud, repose sur Kubernetes : chaque service lancé par un utilisateur (RStudio, Jupyter, VS Code, une base de données) devient un ensemble de

pods dans le cluster.

1.7 Docker en dix commandes

Docker reste l'outil le plus courant pour construire et tester des images sur un poste. La démo du jour 1 enchaîne ces commandes ; refaites-les chez vous avec Docker Desktop, Rancher Desktop ou Podman (mêmes commandes, `podman` au lieu de `docker`).

Commande	Effet
`docker run -d --name web -p 8080:80 nginx:1.27-alpine`	Lance un conteneur en arrière-plan et publie son port 80 sur le port 8080 du poste
`docker ps`	Liste les conteneurs en cours (`-a` : aussi les arrêtés)
`docker logs -f web`	Affiche la sortie de l'application
`docker exec -it web sh`	Ouvre un shell dans le conteneur
`docker stop web` , `docker start web`	Arrête puis relance le même conteneur
`docker rm -f web`	Supprime le conteneur et tout ce qui a été écrit dedans
`docker run -v "$PWD/site:/usr/share/nginx/html:ro" …`	Monte un dossier du poste : les données survivent au conteneur
`docker build -t mon-site:1.0 .`	Construit une image à partir du Dockerfile du dossier courant
`docker history mon-site:1.0`	Montre les couches de l'image
`docker push registre.exemple.fr/equipe/mon-site:1.0`	Publie l'image dans un registre (après `docker tag` et `docker login`)

Un Dockerfile décrit la construction d'une image. Les commentaires y occupent une ligne entière : un `#` en fin d'instruction serait lu comme un argument.

```
# Image de départ
FROM nginx:1.27-alpine
# Ajoute les fichiers du site : une nouvelle couche
COPY site/ /usr/share/nginx/html/
# Documente le port d'écoute ; la commande de démarrage est héritée de
l'image nginx
EXPOSE 80
```

Ce qui fait une bonne image pour Kubernetes :

- une image de départ officielle et minimale (alpine, distroless) ;
- une seule application par image, qui journalise sur la sortie standard ;
- un utilisateur non root (`USER 10001`), exigé par le niveau `restricted` (module 15) ;
- un tag de version explicite, jamais `latest` ;
- aucune donnée sensible dans l'image : configuration et secrets arrivent au démarrage (module 8).

1.8 Sous le capot : containerd sans Docker (TP1)

Les nœuds Kubernetes n'ont pas Docker, seulement containerd. Son outil de bas niveau, `ctr` , permet de vérifier qu'un conteneur n'est qu'un processus de l'hôte.

```
sudo ctr images pull mirror.gcr.io/library/nginx:1.27
sudo ctr run -d --net-host mirror.gcr.io/library/nginx:1.27 web
sudo ctr tasks ls                     # PID du processus principal
ps -o pid,user,cmd -p <PID>           # un processus comme les autres
sudo lsns -p <PID>                    # ses namespaces
sudo ctr tasks exec -t --exec-id shell web sh
#   cat /etc/os-release  -> Debian : l'espace utilisateur vient de l'image
#   uname -r             -> le noyau Ubuntu de l'hôte, partagé
```

Outil	Parle à	Usage
`docker`	Le démon Docker	Poste de développement : construire, tester
`ctr`	containerd directement	Diagnostic de bas niveau, pédagogie
`crictl`	containerd via le CRI, comme le kubelet	Diagnostic d'un nœud Kubernetes (module 4)
`nerdctl`	containerd, avec la syntaxe de Docker	Remplacer Docker sur un hôte containerd

containerd range ses objets dans des espaces de noms : `default` pour vos essais avec `ctr` , `k8s.io` pour ce que lance Kubernetes. Après le TP2, `sudo ctr -n k8s.io containers ls` montre ainsi les conteneurs des pods.

À retenir

- Un conteneur est un processus isolé par les namespaces, limité par les cgroups, et qui voit un système de fichiers en couches.

- Une image OCI est portable entre Docker, containerd et CRI-O ; désignez-la par un tag précis, jamais `latest` en production.
- Kubernetes pilote containerd via le CRI ; il n'a pas besoin du moteur Docker.
- Kubernetes répond aux problèmes qu'un hôte seul ne règle pas : placement, pannes, mise à l'échelle, mises à jour, découverte de services.

Questions de révision

1. Quel mécanisme du noyau empêche un conteneur de consommer toute la mémoire de l'hôte ?
2. Que désigne exactement la référence d'image `redis:7-alpine` ?
3. Pourquoi peut-on exécuter dans Kubernetes une image construite avec Docker, alors que Kubernetes n'utilise pas Docker ?
4. Citez trois problèmes qu'un orchestrateur règle et que Docker Compose ne règle pas.

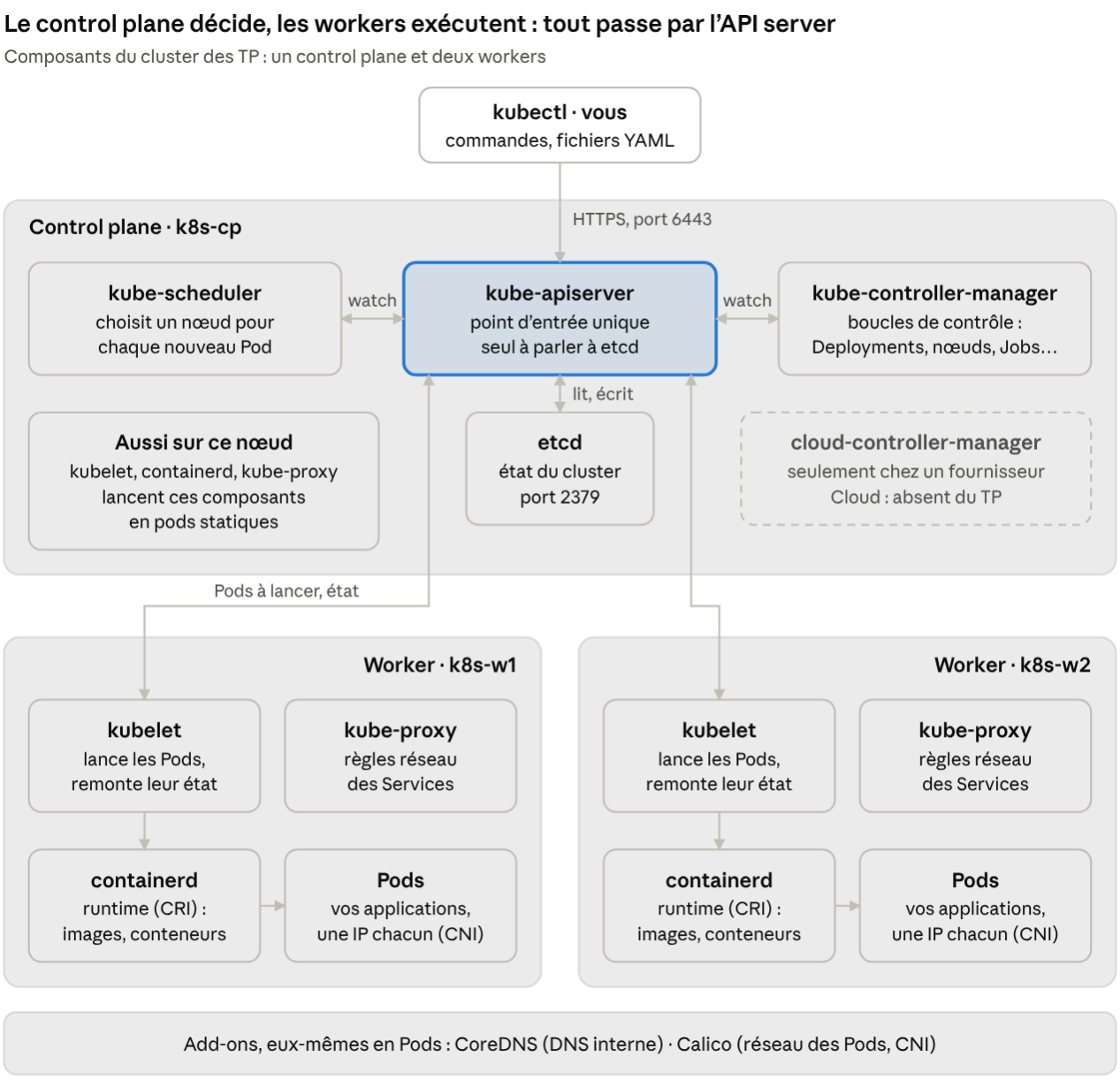
Module 2 — Architecture de Kubernetes

Un cluster se compose d'un control plane, qui décide, et de nœuds de travail, qui exécutent. Tous les composants communiquent uniquement à travers l'API server, et chacun ramène en boucle la réalité vers l'état décrit dans etcd. Comprendre ce schéma permet de diagnostiquer presque toutes les pannes.

2.1 Vue d'ensemble

- Le **control plane** héberge le « cerveau » : l'API, la base de données du cluster, le placement et les boucles de contrôle.
- Les **nœuds de travail** (workers) exécutent les applications sous forme de pods.
- Des **add-ons** complètent l'ensemble : réseau des pods, DNS, métriques.

Dans votre TP, `k8s-cp` porte le control plane et `k8s-w1`, `k8s-w2` sont les workers. En production, on exécute 3 nœuds de control plane derrière un répartiteur de charge pour la haute disponibilité.



architecture du cluster des TP · control plane et workers

Toutes les flèches aboutissent à l'API server : les composants ne s'échangent jamais l'état du cluster directement. Il existe bien quelques liaisons directes : le kubelet pilote containerd par le CRI, l'API server joint le kubelet pour kubectl logs et exec, et lui seul parle à etcd. Les sections 2.2 à 2.4 détaillent chaque boîte.

2.2 Les composants du control plane

Composant	Rôle	Port	Dans le TP
kube-apiserver	Point d'entrée unique : API REST, authentification, autorisation, validation, écriture dans etcd	6443	Pod statique

Composant	Rôle	Port	Dans le TP
etcd	Base clé-valeur qui contient tout l'état du cluster	2379-2380	Pod statique
kube-scheduler	Choisit un nœud pour chaque pod qui n'en a pas	10259	Pod statique
kube-controller-manager	Exécute les boucles de contrôle intégrées	10257	Pod statique
cloud-controller-manager	Fait le lien avec un fournisseur Cloud (répartiteurs, disques, nœuds)	—	Absent

kube-apiserver. Toute action passe par lui : la commande `kubectl`, le scheduler, le kubelet, les contrôleurs. Il vérifie qui vous êtes (authentification), ce que vous avez le droit de faire (autorisation RBAC, module 15), applique les contrôleurs d'admission (valeurs par défaut, politiques), valide l'objet puis l'enregistre dans etcd. C'est le **seul** composant qui parle à etcd. Les autres composants s'abonnent à ses changements par un mécanisme de *watch* : ils sont prévenus dès qu'un objet qui les concerne change.

etcd. Une base clé-valeur distribuée qui utilise l'algorithme de consensus Raft. En haute disponibilité, on déploie un nombre impair de membres (3 ou 5) : le cluster reste inscriptible tant qu'une majorité (le quorum) répond. Tout l'état du cluster y est : perdre etcd sans sauvegarde, c'est perdre le cluster. etcd est sensible à la latence disque, d'où la recommandation de disques SSD.

kube-scheduler. Il surveille les pods sans nœud attribué et choisit le meilleur nœud en deux temps :

1. **Filtrage** : il écarte les nœuds qui ne conviennent pas (ressources insuffisantes, taint non tolérée, affinité non respectée, port déjà pris).
2. **Notation** : il classe les nœuds restants (équilibrage de charge, répartition des réplicas, image déjà présente).

Il inscrit ensuite le nom du nœud dans le pod (`spec.nodeName`). Il ne démarre rien lui-même.

kube-controller-manager. Un seul processus qui fait tourner des dizaines de contrôleurs : Deployment, ReplicaSet, Job, Node (surveille les nœuds qui ne répondent plus), EndpointSlice (tient à jour les adresses derrière chaque Service), ServiceAccount, Namespace, et d'autres.

2.3 Les composants des nœuds

Composant	Rôle	Forme
kubelet	Agent du nœud : fait tourner les pods qui lui sont attribués, exécute les probes, remonte l'état	Service systemd (port 10250)
Runtime de conteneurs	containerd télécharge les images et démarre les conteneurs via runc	Service systemd
kube-proxy	Traduit chaque Service en règles réseau sur le nœud	DaemonSet (un pod par nœud)

kubelet. Ce n'est pas un pod : c'est un service système, car c'est lui qui fait tourner les pods. Il interroge l'API server pour connaître les pods attribués à son nœud, demande à containerd de les créer via le CRI, configure leur réseau via le plugin CNI, surveille leur santé et remonte leur état. Il lance aussi les **pods statiques** décrits dans `/etc/kubernetes/manifests` : c'est ainsi que démarre le control plane créé par kubeadm.

kube-proxy. Il programme sur chaque nœud les règles qui redirigent le trafic d'un Service vers ses pods. Trois modes existent : `iptables` (par défaut), `nftables` (son successeur) et `ipvs` (déprécié). Certains plugins réseau, comme Cilium ou Calico en mode eBPF, peuvent le remplacer.

Le nœud de control plane possède lui aussi un kubelet et containerd : ils lui servent à exécuter les pods statiques du control plane. Une *taint* l'empêche par défaut de recevoir vos applications (module 5).

2.4 Les add-ons indispensables

Le plugin réseau (CNI). Kubernetes définit des règles réseau mais ne les implémente pas :

- chaque pod reçoit sa propre adresse IP ;
- tout pod peut joindre tout autre pod, sur n'importe quel nœud, sans traduction d'adresse (NAT) ;
- les agents d'un nœud (kubelet, par exemple) peuvent joindre tous les pods de ce nœud.

Un plugin CNI met ces règles en œuvre : Calico (le TP), Cilium, Flannel. Sans CNI, les nœuds restent `NotReady` et CoreDNS reste `Pending`.

CoreDNS. Le serveur DNS interne : il traduit le nom d'un Service (`redis.fil-rouge.svc.cluster.local`) en son adresse IP. Il tourne sous forme de Deployment dans `kube-system`.

Autres add-ons courants. metrics-server (métriques CPU et mémoire, module 10), un contrôleur Ingress ou Gateway pour l'entrée HTTP, des pilotes de stockage CSI (module 16).

2.5 Le modèle déclaratif et la réconciliation

C'est l'idée la plus importante de la formation.

- **Impératif** : « lance 3 conteneurs nginx ». Si l'un s'arrête demain, personne ne le relance.
- **Déclaratif** : « je veux en permanence 3 réplicas de nginx ». Le système compare sans cesse ce qui existe à ce qui est demandé, et corrige.

À ne pas confondre avec la section 6.6, où « impératif » désigne la façon de taper la commande (`kubectl create`) et non le fonctionnement. Un Deployment créé par `kubectl create deployment` est un état désiré comme un autre : ses contrôleurs le réconcilient en permanence.

Chaque objet porte donc deux parties : `spec`, l'état désiré que vous écrivez, et `status`, l'état observé que Kubernetes remplit. Chaque contrôleur exécute la même boucle :

1. **Observer** l'état réel (via l'API server).
2. **Comparer** avec l'état désiré.
3. **Agir** pour réduire l'écart, puis recommencer.

Une analogie : le thermostat. Vous réglez 19 °C ; il ne « lance pas le chauffage pendant 2 heures », il mesure et corrige en continu. C'est pourquoi un pod supprimé à la main est aussitôt remplacé : le contrôleur du ReplicaSet constate 2 pods au lieu de 3 et en recrée un. Pour arrêter un pod durablement, on modifie l'état désiré (`replicas: 0`), pas le pod.

2.6 Parcours d'un `kubectl apply`

Voici ce qui se passe quand vous appliquez un Deployment de 3 réplicas :

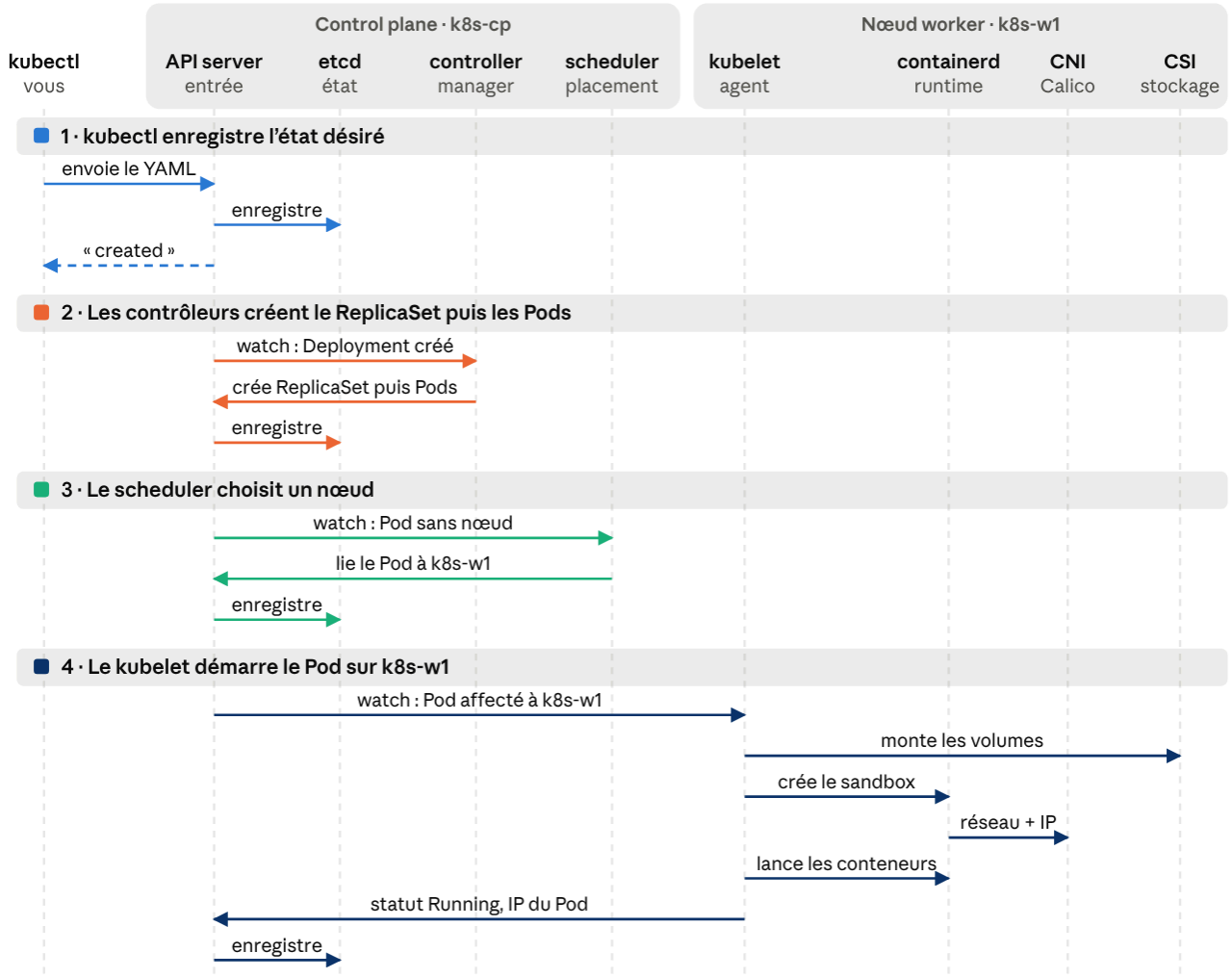
1. `kubectl` lit votre fichier `~/.kube/config` et envoie une requête HTTPS à l'API server.
2. L'API server authentifie la requête (certificat client), vérifie vos droits (RBAC), passe l'objet aux contrôleurs d'admission, le valide et l'enregistre dans etcd.
3. Le contrôleur Deployment, prévenu par le watch, crée un objet ReplicaSet.
4. Le contrôleur ReplicaSet crée 3 objets Pod, encore sans nœud.
5. Le scheduler voit 3 pods non placés, choisit un nœud pour chacun et l'inscrit dans le pod.
6. Le kubelet du nœud choisi voit son pod. Il fait monter les volumes (CSI), demande à containerd de créer le sandbox du pod, dont le plugin CNI configure le réseau et l'adresse IP, puis de télécharger l'image et de démarrer les conteneurs.

7. Le kubelet remonte l'état `Running`. Si un Service sélectionne ces pods, le contrôleur EndpointSlice ajoute leurs adresses et kube-proxy met à jour ses règles.

Aucun composant n'a transmis l'état directement à un autre : tous ont lu et écrit dans l'API server. La seule liaison directe est locale au nœud : le kubelet a demandé à containerd de lancer les conteneurs. Ce couplage faible rend le système robuste et extensible. Le diagramme ci-dessous reprend ce parcours, phase par phase.

kubectl rend la main tout de suite ; chaque composant réagit ensuite par watch

Parcours d'un kubectl apply de Deployment, jusqu'au Pod en état Running



kubectl apply · 16 messages en 4 phases

Chaque phase commence par un watch : le composant concerné est prévenu par l'API server, agit, puis écrit son résultat dans l'API. Sur le nœud, l'ordre est fixe : les volumes d'abord, puis le sandbox et son réseau, enfin les conteneurs.

2.7 Les objets de l'API

Tout, dans Kubernetes, est un objet de l'API, identifié par un `kind` (Pod, Deployment…) et une `apiVersion` (groupe et version).

- **Groupes** : le groupe historique « core » s'écrit `v1` (Pod, Service, ConfigMap) ; les autres s'écrivent `groupe/version` : `apps/v1` (Deployment), `batch/v1` (Job), `networking.k8s.io/v1` (NetworkPolicy).
- **Maturité** : `v1alpha1` (expérimental, désactivé par défaut), `v1beta1` (testé, peut encore changer), `v1` (stable).
- **Portée** : la plupart des objets vivent dans un **namespace** (Pod, Service, Deployment) ; d'autres valent pour tout le cluster (Node, PersistentVolume, Namespace lui-même).

```
kubectl api-resources                      # tous les types, avec leur groupe
et leur portée
kubectl api-resources --namespaced=false   # les types de portée cluster
kubectl explain deployment                 # documentation intégrée
```

2.8 Observer l'architecture sur votre cluster

```
kubectl get nodes -o wide                    # nœuds, versions, runtime
kubectl get pods -n kube-system -o wide      # composants et add-ons
ls /etc/kubernetes/manifests                 # [cp] les 4 pods statiques
systemctl status kubelet                     # [tous] le kubelet est un
service
sudo crictl ps                               # [tous] conteneurs vus par
containerd
kubectl get --raw='/readyz?verbose'          # état détaillé de l'API server
```

Si le control plane tombe. Les applications déjà lancées continuent de tourner : les kubelets et kube-proxy gardent leur dernière configuration. En revanche, plus aucune modification, plus de placement, plus d'auto-réparation. Votre cluster de TP n'a qu'un control plane : c'est un point unique de défaillance, acceptable pour apprendre, pas en production.

À retenir

- L'API server est le centre : seul à parler à etcd, tous les autres composants passent par lui.
- Le scheduler choisit un nœud ; le kubelet démarre réellement les conteneurs.
- Vous décrivez un état désiré (`spec`) ; des contrôleurs le réconcilient en boucle avec l'état réel (`status`).
- etcd contient tout le cluster : il se sauvegarde et se déploie en nombre impair en production.

Questions de révision

1. Pourquoi le kubelet est-il un service systemd et non un pod ?
2. Un pod reste en `Pending` : quel composant n'a pas fait son travail, et pourquoi peut-il refuser ?
3. Vous supprimez un pod d'un Deployment : quels composants interviennent, dans quel ordre ?
4. Que se passe-t-il pour les applications si le nœud de control plane s'arrête ?

Module 3 — Kubernetes dans l'écosystème Cloud

Le Kubernetes que vous installez avec kubeadm est le même que celui des offres Cloud : seule change la répartition des responsabilités. Ce module situe les façons d'obtenir un cluster et les outils qui gravitent autour.

3.1 Trois façons d'obtenir un cluster

Mode	Qui gère le control plane	Exemples	Adapté à
Service managé	Le fournisseur Cloud	Google GKE, Amazon EKS, Microsoft AKS ; en France OVHcloud Managed Kubernetes, Scaleway Kapsule	Équipes qui veulent se concentrer sur les applications
Distribution installée	Vous (outillé par l'éditeur)	Red Hat OpenShift, SUSE Rancher (RKE2), k3s, Talos Linux, ou kubeadm directement	Centres de données internes, besoins de maîtrise ou de souveraineté
Cluster local	Vous, sur votre poste	minikube, kind, k3d, Docker Desktop, Rancher Desktop	Développement, tests, apprentissage

Dans le secteur public, le choix dépend aussi de la sensibilité des données : la doctrine « cloud au centre » de l'État et la qualification SecNumCloud de l'ANSSI orientent vers des offres de confiance ou des infrastructures internes.

3.2 Managé ou auto-hébergé : qui fait quoi

Responsabilité	Service managé	Cluster auto-hébergé (kubeadm, distribution)
Haute disponibilité du control plane	Fournisseur	Vous : 3 nœuds, répartiteur de charge
Sauvegarde d'etcd	Fournisseur	Vous
Mises à jour de Kubernetes	Un clic ou automatiques	Vous, version mineure par version mineure
Service de type LoadBalancer	Fourni (répartiteur Cloud)	À ajouter (MetalLB par exemple)
Stockage dynamique	Disques Cloud prêts à l'emploi	À ajouter (Ceph, NFS, pilote CSI de votre baie)
Ajout automatique de nœuds	Disponible	Difficile sans couche de virtualisation pilotable
Applications, sécurité des workloads	Vous	Vous

Dans tous les cas, ce que vous déployez dans le cluster (manifestes, RBAC, quotas, NetworkPolicies) reste votre responsabilité. Tout ce que vous apprenez à partir du module 6 s'applique à l'identique sur un service managé.

3.3 Pourquoi apprendre avec kubeadm

- C'est l'outil officiel du projet pour créer un cluster conforme, et la brique sur laquelle reposent de nombreux outils (Cluster API, plusieurs distributions).
- Il expose ce que les offres managées cachent : certificats, pods statiques, etcd, plugin réseau. Savoir ce qu'il y a sous le capot aide à diagnostiquer, même sur un cluster managé.
- Il ne fait que l'essentiel : pas de répartiteur de charge, pas de stockage, pas de supervision. Vous verrez ce qu'il faut ajouter.

3.4 L'écosystème CNCF

La CNCF héberge des centaines de projets complémentaires. Les plus courants :

Besoin	Projets
Réseau des pods et politiques réseau	Calico, Cilium, Flannel
Entrée HTTP	Contrôleurs Ingress, implémentations de la Gateway API (Envoy Gateway, Traefik, Istio)
Paquets et modèles	Helm, Kustomize
Déploiement continu (GitOps)	Argo CD, Flux
Métriques et alertes	Prometheus, Grafana, OpenTelemetry
Logs	Loki, Fluent Bit
Stockage	Rook (Ceph), Longhorn, pilotes CSI
Sécurité et conformité	cert-manager, Kyverno, OPA Gatekeeper, Falco, Trivy
Montée en charge	KEDA, Karpenter, Cluster Autoscaler
Registre d'images	Harbor

Le programme *Certified Kubernetes* de la CNCF garantit qu'une distribution ou une offre passe les tests de conformité : un manifeste qui fonctionne sur l'une fonctionne sur l'autre, sauf extensions propres à un fournisseur.

3.5 Les clusters locaux

Pour expérimenter sur un poste, sans VM dédiées :

```
# minikube : un cluster d'un nœud dans un conteneur ou une VM
minikube start --cpus=2 --memory=4g
kubectl get nodes
minikube delete

# kind : Kubernetes dans des conteneurs Docker, idéal pour l'intégration
continue
kind create cluster --name essai
kubectl cluster-info --context kind-essai
kind delete cluster --name essai
```

Ces outils conviennent au développement et aux tests de manifestes ; ils ne remplacent pas un vrai cluster pour l'exploitation, le réseau multi-nœuds ou la mesure de performances.

3.6 Interfaces graphiques

- **Kubernetes Dashboard**, l'ancienne interface web officielle, a été archivé le 21 janvier 2026 faute de mainteneurs ; il ne reçoit plus de correctifs de sécurité (source).
- **Headlamp** est son successeur recommandé : interface web extensible, qui respecte les droits RBAC de l'utilisateur. Dans nos TP, la console web ne donne pas de navigateur vers les VM : vous la verrez en démonstration.
- **k9s** offre une interface en mode texte dans le terminal, très appréciée des administrateurs : c'est celle que vous utiliserez au TP9. **Lens** et son fork libre **FreeLens** sont des applications de bureau.

Une interface aide à explorer ; `kubectl` et les fichiers YAML restent la référence, car eux seuls se versionnent et se rejouent.

3.7 Pour aller plus loin : les certifications

La Linux Foundation et la CNCF proposent des certifications pratiques, passées en ligne sur un vrai cluster : KCNA (notions de base), CKAD (développeur d'applications), CKA (administrateur), CKS (sécurité). Cette formation couvre une bonne partie du programme CKA.

À retenir

- Un service managé prend en charge le control plane, pas vos applications ni leur sécurité.
- Les manifestes Kubernetes sont portables d'un cluster conforme à l'autre.
- kubeadm est l'outil de référence pour comprendre et monter un cluster ; il faut lui ajouter réseau, stockage et supervision.
- Le Kubernetes Dashboard est archivé depuis janvier 2026 ; utilisez Headlamp (web) ou k9s (terminal).

Questions de révision

1. Citez deux responsabilités qui restent les vôtres sur un service Kubernetes managé.
2. Pourquoi un Service de type LoadBalancer reste-t-il en attente sur un cluster kubeadm sans ajout ?
3. Dans quel cas préférer kind ou minikube à un cluster de VM ?

Module 4 — Installer un cluster avec kubeadm

Installer un cluster tient en quatre étapes : préparer les nœuds, installer le runtime et les binaires Kubernetes, initialiser le control plane, puis ajouter le réseau et les workers. Ce module explique ce que fait chaque commande des TP1 et TP2, pour que vous sachiez réparer quand quelque chose échoue.

4.1 Le cluster cible

Nœud	Rôle	Exemple d'IP
`k8s-cp`	Control plane (API server, etcd, scheduler, controller-manager)	10.10.0.10
`k8s-w1`	Worker	10.10.0.20
`k8s-w2`	Worker	10.10.0.30

Trois plages d'adresses coexistent et ne doivent jamais se chevaucher : le réseau des VM (ici 10.10.0.0/24), le réseau des pods (10.244.0.0/16) et le réseau des Services (10.96.0.0/12).

4.2 Prérequis système et leur raison d'être

Prérequis	Pourquoi
2 vCPU et 2 Go de RAM au minimum sur le control plane	kubeadm le vérifie au démarrage et refuse en dessous
Nom d'hôte, adresse MAC et identifiant machine uniques	Le nom d'hôte devient le nom du nœud dans le cluster
Swap désactivé	Par défaut le kubelet refuse de démarrer avec du swap actif ; le support du swap existe mais demande une configuration explicite
Modules `overlay` et `br_netfilter`	`overlay` sert au système de fichiers en couches ; `br_netfilter` permet aux règles iptables de voir le trafic des ponts réseau
`net.ipv4.ip_forward = 1`	Le nœud doit router les paquets entre pods et vers l'extérieur
`net.bridge.bridge-nf-call-iptables = 1`	Les règles des Services (kube-proxy) doivent s'appliquer au trafic ponté

Prérequis	Pourquoi
Ports ouverts entre nœuds	6443 (API), 2379-2380 (etcd), 10250 (kubelet), 10257 et 10259 (contrôleurs, scheduler), 30000-32767 (NodePort), plus ceux du CNI
Horloge synchronisée (NTP)	Les certificats ont des dates de validité ; une horloge décalée les fait rejeter
Runtime compatible CRI	Kubernetes ne lance pas les conteneurs lui-même

4.3 Le runtime : containerd 2

Kubernetes 1.35 a été la dernière version à prendre officiellement en charge containerd 1.x ; depuis la 1.36, containerd 2.0 ou plus est requis. Techniquement, containerd 1.7 démarre encore avec un kubelet 1.37, mais cette compatibilité disparaît en 1.38 (source). Le paquet `containerd` d'Ubuntu 24.04 est encore en 1.7 : on installe donc le paquet `containerd.io` du dépôt Docker, qui fournit la branche 2.x.

Après installation, on régénère la configuration par défaut, car celle du paquet désactive l'interface CRI dont Kubernetes a besoin :

```
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
sudo systemctl restart containerd
```

Pourquoi `SystemdCgroup = true` ? Sur une machine dont systemd est le gestionnaire de services, il ne doit y avoir qu'un seul gestionnaire des cgroups. Le kubelet et le runtime doivent utiliser le même pilote, `systemd` ; sinon le nœud devient instable sous charge. Avec containerd 2, le kubelet détecte automatiquement le pilote utilisé par le runtime.

crictl est l'outil de diagnostic côté nœud. Il parle directement au runtime, sans passer par l'API server, et fonctionne donc même si le control plane est en panne. Il est fourni par le paquet `cri-tools` du dépôt pkgs.k8s.io, à installer explicitement : les paquets kubeadm récents ne l'installent plus. On lui indique une fois le socket de containerd avec `sudo crictl config --set runtime-endpoint=unix:///run/containerd/containerd.sock`, puis :

```
sudo crictl ps                # conteneurs en cours
sudo crictl pods              # sandboxes de pods
sudo crictl images            # images présentes sur le nœud
sudo crictl logs <id>         # logs d'un conteneur
```

4.4 Les trois binaires Kubernetes

Binaire	Rôle	Où
`kubeadm`	Crée, joint, met à jour et réinitialise un cluster	Tous les nœuds
`kubelet`	Agent qui fait tourner les pods	Tous les nœuds
`kubectl`	Client en ligne de commande de l'API	Là où vous administrez (ici `k8s-cp`)

Ils viennent du dépôt `pkgs.k8s.io`, qui propose **un dépôt par version mineure** (`/core:/stable:/v1.37/deb/`). Pour passer en 1.38, il faudra changer l'adresse du dépôt. La commande `apt-mark hold` bloque les paquets : une mise à jour système automatique ne doit jamais changer la version de Kubernetes sans procédure.

> **TP1 — à faire maintenant** · Jour 1, 10:50 · 70 min
>
> Préparer les 3 nœuds en appliquant les sections 4.2 à 4.4 sur `k8s-cp`, puis manipuler un premier conteneur avec `ctr` (section 1.8). Un script prépare ensuite les deux workers.
>
> **Point de contrôle :** containerd en 2.x, kubeadm en v1.37.x, swap désactivé, `ip_forward` à 1.

4.5 Ce que fait `kubeadm init`

```
sudo kubeadm init --apiserver-advertise-address=10.10.0.10 --pod-network-cidr=10.244.0.0/16
```

La commande enchaîne des phases que l'on peut aussi lancer une par une (`kubeadm init phase --help`) :

Phase	Ce qu'elle fait	Résultat
`preflight`	Vérifie les prérequis, télécharge les images	Erreurs bloquantes si swap, ports occupés, ip_forward…

Phase	Ce qu'elle fait	Résultat
`certs`	Crée les autorités de certification et les certificats	`/etc/kubernetes/pki/`
`kubeconfig`	Crée les fichiers d'accès des composants et de l'administrateur	`/etc/kubernetes/*.conf`
`kubelet-start`	Écrit la configuration du kubelet et le démarre	`/var/lib/kubelet/config.yaml`
`control-plane`, `etcd`	Écrit les manifestes des pods statiques	`/etc/kubernetes/manifests/`
`upload-config`	Enregistre la configuration du cluster dans l'API	ConfigMap `kubeadm-config`
`mark-control-plane`	Ajoute le label et la taint du control plane	Les applications évitent ce nœud
`bootstrap-token`	Crée le jeton qui permettra aux workers de rejoindre	Valable 24 h
`addon`	Installe CoreDNS et kube-proxy	Deployment et DaemonSet dans `kube-system`

Principales options :

Option	Rôle	Défaut
`--apiserver-advertise-address`	Adresse sur laquelle l'API server écoute et se fait connaître	Adresse de la route par défaut
`--pod-network-cidr`	Plage des adresses de pods, à accorder avec le CNI	Aucune
`--service-cidr`	Plage des adresses de Services	10.96.0.0/12
`--control-plane-endpoint`	Nom ou IP stable du répartiteur, indispensable pour la haute disponibilité	Aucun
`--upload-certs`	Partage les certificats pour ajouter d'autres control planes	Désactivé

En production, on préfère un fichier de configuration versionné : `sudo kubeadm init --config kubeadm.yaml`. La commande `kubeadm config print init-defaults` en donne un modèle complet.

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 10.10.0.10
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
networking:
  podSubnet: 10.244.0.0/16
  serviceSubnet: 10.96.0.0/12
```

4.6 La PKI du cluster

Les communications entre les composants de Kubernetes (API server, etcd, kubelets, scheduler, controller-manager, kubectl) sont chiffrées et authentifiées par certificats TLS. Le trafic de vos applications ne l'est pas automatiquement : entre podinfo et Redis, il circule en clair. kubeadm crée plusieurs autorités dans `/etc/kubernetes/pki/` :

- `ca.crt` / `ca.key` : l'autorité principale, qui signe les certificats de l'API server, des kubelets et des administrateurs ;
- `etcd/ca.crt` : une autorité dédiée à etcd ;
- `front-proxy-ca.crt` : pour les extensions de l'API ;
- `sa.key` / `sa.pub` : la paire de clés qui signe les jetons des ServiceAccounts.

Les autorités sont valables 10 ans, les certificats des composants **1 an**. Un cluster jamais mis à jour cesse de fonctionner au bout d'un an. Les mises à jour avec `kubeadm upgrade` renouvellent les certificats ; sinon :

```
sudo kubeadm certs check-expiration    # dates d'expiration
sudo kubeadm certs renew all           # renouvellement, puis redémarrer les
pods statiques
```

4.7 Le fichier kubeconfig

`kubectl` sait à quel cluster parler, et sous quelle identité, grâce à un fichier kubeconfig (par défaut `~/.kube/config`, ou la variable `KUBECONFIG`). Il contient trois listes reliées entre elles :

```
apiVersion: v1
kind: Config
clusters:
  - name: kubernetes
    cluster:
      server: https://10.10.0.10:6443
      certificate-authority-data: LS0t...      # CA du cluster, en base64
users:
  - name: kubernetes-admin
    user:
      client-certificate-data: LS0t...         # identité de l'administrateur
      client-key-data: LS0t...
contexts:
  - name: kubernetes-admin@kubernetes
    context:
      cluster: kubernetes
      user: kubernetes-admin
      namespace: fil-rouge                     # namespace par défaut
(optionnel)
current-context: kubernetes-admin@kubernetes
```

```
kubectl config get-contexts
kubectl config use-context <contexte>
kubectl config set-context --current --namespace=fil-rouge
```

kubeadm produit `admin.conf` (administrateur du cluster) et `super-admin.conf` (accès de secours qui contourne RBAC). Ces fichiers donnent tous les droits : ils ne se copient pas sur un poste et ne se partagent pas. Pour déléguer, on crée des identités limitées par RBAC (module 15).

4.8 Les pods statiques

Le control plane créé par kubeadm tourne sous forme de **pods statiques** : des fichiers YAML déposés dans `/etc/kubernetes/manifests/`, que le kubelet lit et exécute directement, sans passer par l'API server. C'est ce qui résout le problème de l'œuf et de la poule : l'API server ne peut pas se créer lui-même via l'API.

- Ils apparaissent dans `kubectl get pods -n kube-system` (suffixés du nom du nœud : `kube-apiserver-k8s-cp`), mais ce sont des reflets : les modifier ou les supprimer via `kubectl` est sans effet durable.
- Pour changer une option de l'API server, on édite le fichier `/etc/kubernetes/manifests/kube-apiserver.yaml` ; le kubelet détecte le changement et redémarre le pod.
- Retirer temporairement le fichier du répertoire arrête le composant.

4.9 Joindre les workers

```
sudo kubeadm join 10.10.0.10:6443 --token abcdef.0123456789abcdef --
discovery-token-ca-cert-hash sha256:<empreinte>
```

- Le **jeton** (*bootstrap token*) authentifie le nouveau nœud auprès du cluster. Il expire après 24 heures.
- L'**empreinte de la CA** permet au nœud de vérifier qu'il parle au bon cluster, et non à un imposteur.
- Le kubelet demande ensuite son propre certificat à l'API (*TLS bootstrapping*) ; la demande est approuvée automatiquement.

```
kubeadm token list                                # [cp] jetons existants
kubeadm token create --print-join-command         # [cp] nouveau jeton et
commande complète
```

Pour ajouter un control plane supplémentaire, on ajoute `--control-plane --certificate-key <clé>` à la commande, après un `kubeadm init` lancé avec `--control-plane-endpoint` et `--upload-certs`.

4.10 Le réseau des pods avec Calico

Après `kubeadm init`, les nœuds restent `NotReady` : aucun plugin réseau n'est installé. Calico va :

1. **Attribuer les adresses** : il découpe 10.244.0.0/16 en blocs (des /26 de 64 adresses) qu'il distribue aux nœuds.
2. **Router entre nœuds** : soit directement, soit en encapsulant les paquets des pods dans des paquets entre nœuds.
3. **Appliquer les NetworkPolicies** (module 15).

Mode d'encapsulation	Principe	Quand l'utiliser
Aucun (routage BGP direct)	Les nœuds s'annoncent les routes des pods	Réseau que vous maîtrisez, sans filtrage d'adresses
IP-in-IP	Paquet IP dans un paquet IP	Réseaux qui laissent passer le protocole IP 4
VXLAN	Trame dans un paquet UDP (port 4789)	Clouds publics et réseaux filtrés : choix du TP

Mode d'encapsulation	Principe	Quand l'utiliser
VXLANCrossSubnet	VXLAN seulement entre sous-réseaux différents	Défaut de Calico ; peut être bloqué par l'anti-usurpation d'adresse des Clouds

Dans le TP, Calico est installé par son **opérateur** (Tigera) : on installe ses définitions de ressources, puis l'opérateur, puis un objet `Installation` qui décrit le réseau voulu. L'opérateur déploie et maintient Calico à partir de cet objet : un premier exemple d'opérateur, détaillé au module 13.

```
kubectl get tigerastatus                # état global de Calico
kubectl get pods -n calico-system -o wide
kubectl get installation default -o yaml # configuration réseau déclarée
```

CNI	Points forts	NetworkPolicy
Flannel	Très simple	Non
Calico	Mature, BGP, VXLAN, eBPF en option	Oui, plus ses propres politiques étendues
Cilium	eBPF, observabilité réseau, peut remplacer kube-proxy	Oui, jusqu'à la couche 7

4.11 Vérifier l'installation

```
kubectl get nodes -o wide            # 3 nœuds Ready, version v1.37.x,
runtime containerd://2.x
kubectl get pods -A                  # tout en Running
kubectl run test-dns --image=mirror.gcr.io/library/busybox:1.36 --rm -it --
restart=Never -- nslookup kubernetes.default
```

> **TP2 — à faire maintenant** · Jour 1, 13:15 · 75 min
>
> Initialiser le cluster avec `kubeadm init`, configurer `kubectl`, installer Calico, puis joindre les deux workers (sections 4.5 à 4.11).
>
> **Point de contrôle :** 3 nœuds `Ready`, pods de `kube-system` et `calico-system` en `Running`, le DNS interne répond.

4.12 Réinitialiser, sauvegarder, mettre à jour

Réinitialiser un nœud (après une erreur d'initialisation, ou pour recommencer) :

```
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d $HOME/.kube/config
```

Sauvegarder etcd : le binaire `etcdctl` est présent dans le pod etcd.

```
kubectl -n kube-system exec etcd-k8s-cp -- etcdctl --
endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt --
cert=/etc/kubernetes/pki/etcd/server.crt --
key=/etc/kubernetes/pki/etcd/server.key snapshot save
/var/lib/etcd/sauvegarde.db
# le fichier se trouve sur k8s-cp dans /var/lib/etcd/ : copiez-le hors du
nœud
```

Mettre à jour vers la version mineure suivante, dans cet ordre :

1. Sur le control plane : pointer le dépôt vers la nouvelle version, mettre à jour `kubeadm`, lancer `sudo kubeadm upgrade plan` puis `sudo kubeadm upgrade apply v1.x.y`.
2. Vider le nœud (`kubectl drain`), mettre à jour `kubelet` et `kubectl`, redémarrer le kubelet, remettre le nœud en service (`kubectl uncordon`).
3. Pour chaque worker : mettre à jour `kubeadm`, lancer `sudo kubeadm upgrade node`, puis drain, mise à jour du kubelet, uncordon.

À retenir

- kubeadm vérifie les prérequis, crée la PKI, écrit les pods statiques du control plane et installe CoreDNS et kube-proxy ; le réseau des pods est à ajouter.
- Le kubelet et containerd doivent utiliser le même pilote de cgroups : `systemd`.
- Les certificats des composants expirent au bout d'un an : mettez à jour régulièrement ou renouvelez-les.
- `admin.conf` donne tous les droits ; on ne le distribue pas.
- Sauvegardez etcd : c'est tout votre cluster.

Questions de révision

1. Pourquoi les nœuds sont-ils `NotReady` juste après `kubeadm init` ?
2. Quel est le rôle de l'empreinte `--discovery-token-ca-cert-hash` dans la commande de join ?
3. Comment modifier une option de démarrage de l'API server sur un cluster kubeadm ?

4. Pourquoi bloque-t-on les paquets `kubelet`, `kubeadm` et `kubectl` avec `apt-mark hold` ?
5. Pourquoi avoir choisi l'encapsulation VXLAN plutôt que le routage direct dans les TP ?

Module 5 — Gérer les nœuds

Un nœud se décrit par ses labels, se protège par des taints, se vide avant maintenance avec `drain` et se remet en service avec `uncordon`. Ce module présente aussi `kubectl`, l'outil que vous utiliserez pendant trois jours.

5.1 kubectl, la boîte à outils

Toutes les commandes suivent la même grammaire : `kubectl <verbe> <type> [<nom>] [options]`.

Verbe	Usage	Exemple
`get`	Lister des objets	`kubectl get pods -o wide`
`describe`	Détail lisible, avec les événements	`kubectl describe node k8s-w1`
`apply`	Créer ou mettre à jour depuis un fichier	`kubectl apply -f app.yaml`
`create`	Créer (échoue si l'objet existe)	`kubectl create namespace test`
`delete`	Supprimer	`kubectl delete pod web-1`
`edit`	Modifier en direct dans un éditeur	`kubectl edit deploy web`
`logs`	Journaux d'un conteneur	`kubectl logs web-1 -f`
`exec`	Commande dans un conteneur	`kubectl exec -it web-1 -- sh`
`explain`	Documentation d'un champ	`kubectl explain pod.spec.tolerations`
`label`, `annotate`	Ajouter ou retirer des métadonnées	`kubectl label node k8s-w2 disque=ssd`
`cordon`, `drain`, `uncordon`	Maintenance des nœuds	`kubectl drain k8s-w2 --ignore-daemonsets`

Options utiles à connaître :

- `-n <namespace>` cible un namespace, `-A` tous les namespaces.
- `-o wide` ajoute des colonnes (IP, nœud) ; `-o yaml` ou `-o json` affiche l'objet complet ; `-o name` ne garde que les noms.
- `-o jsonpath='{.items[*].metadata.name}'` et `-o custom-columns=NOM:.metadata.name,NOEUD:.spec.nodeName` extraient des champs précis.
- `-w` (watch) suit les changements en direct ; `-l app=web` filtre par label ; `--show-labels` affiche les labels.
- Les types ont des noms courts : `po` (pods), `deploy`, `svc`, `ns`, `no` (nodes), `cm` (configmaps), `pvc`. Liste complète : `kubectl api-resources`.

Confort indispensable, dans `~/.bashrc` :

```
source <(kubectl completion bash)
alias k=kubectl
complete -o default -F __start_kubectl k
```

5.2 Lire l'état d'un nœud

`kubectl describe node k8s-w1` affiche, dans l'ordre :

- **Labels et annotations** : identité du nœud (nom, OS, architecture, rôle).
- **Taints** : ce que le nœud repousse.
- **Conditions** : `Ready` (le kubelet répond et le nœud peut accueillir des pods), `MemoryPressure`, `DiskPressure`, `PIDPressure`. Toute condition de pression à `True` déclenche des évictions.
- **Capacity et Allocatable** : la capacité brute, et ce qui reste réellement disponible pour les pods une fois retirées les réserves du système et du kubelet.
- **Allocated resources** : la somme des `requests` et `limits` des pods placés. C'est ce chiffre, et non la consommation réelle, que regarde le scheduler.
- **Events** : les derniers événements du nœud.

Si un nœud ne répond plus, le contrôleur Node le passe `NotReady` au bout de quelques dizaines de secondes et lui applique une taint `node.kubernetes.io/unreachable`. Par défaut, ses pods tolèrent cette taint pendant 300 secondes ; passé ce délai, ils sont évincés et leurs contrôleurs les recréent ailleurs. Une panne de nœud coûte donc environ 5 minutes d'indisponibilité aux pods qui n'ont qu'un réplica.

5.3 Les labels des nœuds

Chaque nœud porte des labels posés automatiquement :

Label	Exemple
`kubernetes.io/hostname`	`k8s-w1`
`kubernetes.io/os`	`linux`
`kubernetes.io/arch`	`amd64`
`node-role.kubernetes.io/control-plane`	présent sur `k8s-cp`, valeur vide
`topology.kubernetes.io/zone`, `.../region`	posés par les fournisseurs Cloud

Vous ajoutez les vôtres pour décrire le matériel ou l'usage :

```
kubectl label node k8s-w2 disque=ssd
kubectl label node k8s-w1 node-role.kubernetes.io/worker=   # affiché dans la
colonne ROLES
kubectl get nodes -L disque
kubectl label node k8s-w2 disque-                           # le tiret final
retire le label
```

5.4 Choisir où tournent les pods

Par défaut, le scheduler place les pods où il veut. Quatre mécanismes l'orientent, du plus simple au plus fin.

nodeSelector : le pod ne va que sur les nœuds qui portent tous les labels indiqués.

```
spec:
  nodeSelector:
    disque: ssd
```

Affinité de nœud : même idée, avec des opérateurs (`In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt`, `Lt`) et deux niveaux d'exigence. Le nom très long rappelle que la règle s'applique au placement, pas aux pods déjà lancés.

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:      # obligatoire
        nodeSelectorTerms:
          - matchExpressions:
              - key: disque
                operator: In
                values: ["ssd", "nvme"]
      preferredDuringSchedulingIgnoredDuringExecution:     # souhaité
        - weight: 50
          preference:
            matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["zone-a"]
```

Affinité et anti-affinité de pods : placer un pod près d'autres pods, ou loin d'eux. L'usage le plus courant : répartir les réplicas d'une application sur des nœuds différents, pour qu'une panne de nœud ne les emporte pas tous.

```
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            topologyKey: kubernetes.io/hostname   # "loin" = sur un autre
nœud
            labelSelector:
              matchLabels:
                app: podinfo
```

Contraintes de répartition (*topology spread constraints*) : imposer un écart maximal du nombre de réplicas entre nœuds ou zones. Plus précis que l'anti-affinité quand il y a plus de réplicas que de nœuds.

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: podinfo
```

5.5 Taints et tolérances

Une **taint** sur un nœud repousse les pods ; une **tolérance** sur un pod l'autorise à ignorer une taint précise. Une tolérance n'attire pas le pod vers le nœud : pour dédier des nœuds à une application, on combine une taint (qui écarte les autres) et une affinité (qui attire celle-ci).

Effet	Comportement
`NoSchedule`	Aucun nouveau pod sans tolérance ; les pods déjà présents restent
`PreferNoSchedule`	Le scheduler évite le nœud s'il a le choix
`NoExecute`	Les pods sans tolérance sont aussi expulsés s'ils y tournent déjà

```
kubectl taint nodes k8s-w2 dedie=calcul:NoSchedule     # poser
kubectl describe node k8s-w2 | grep Taints
kubectl taint nodes k8s-w2 dedie=calcul:NoSchedule-    # retirer (tiret
final)
```

```
spec:
  tolerations:
    - key: dedie
      operator: Equal
      value: calcul
      effect: NoSchedule
```

Vous avez déjà rencontré deux taints : `node-role.kubernetes.io/control-plane:NoSchedule`, qui tient vos applications à l'écart du control plane, et `node.kubernetes.io/not-ready`, posée tant qu'un nœud n'a pas de réseau. Les DaemonSets système (Calico, kube-proxy) tolèrent ces taints : c'est pourquoi ils tournent partout.

5.6 Maintenance d'un nœud

Commande	Effet
`kubectl cordon k8s-w2`	Marque le nœud non planifiable ; les pods présents restent
`kubectl drain k8s-w2 --ignore-daemonsets --delete-emptydir-data`	Cordon, puis éviction de tous les pods gérés par un contrôleur

Commande	Effet
`kubectl uncordon k8s-w2`	Remet le nœud en service

Options de `drain` :

- `--ignore-daemonsets` : les pods de DaemonSet ne sont pas évincés (ils reviendraient aussitôt) ;
- `--delete-emptydir-data` : accepte de perdre les volumes temporaires `emptyDir` ;
- `--force` : évince aussi les pods créés sans contrôleur, qui ne seront recréés nulle part ;
- `--timeout=5m` : abandonne si l'éviction traîne.

Un **PodDisruptionBudget** protège une application pendant les maintenances : `drain` attend plutôt que de descendre sous le minimum fixé.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: podinfo-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: podinfo
```

Procédure type pour redémarrer ou mettre à jour un nœud : `drain` → intervention (mises à jour système, redémarrage) → vérifier `Ready` → `uncordon`. Un nœud à la fois.

Retirer définitivement un worker : `kubectl drain`, puis `kubectl delete node k8s-w2`, puis `sudo kubeadm reset -f` sur le nœud lui-même.

5.7 Capacité et éviction

La mémoire allouable d'un nœud est sa mémoire totale moins les réserves du système et du kubelet, et moins un seuil d'éviction (100 Mi par défaut). Quand la mémoire disponible passe sous ce seuil, le kubelet évince des pods, en commençant par ceux qui consomment le plus au-delà de leurs `requests`. D'où l'importance de déclarer des ressources réalistes (module 11).

> **TP3 — à faire maintenant** · Jour 1, 14:35 · 60 min
>
> Étiqueter les nœuds, poser puis retirer une taint, vider et remettre en service un worker, déployer et exposer un premier nginx (sections 5.1 à 5.6).
>
> **Point de contrôle :** la page nginx s'affiche via le NodePort ; `k8s-w2` est de nouveau planifiable.
>
> Fin du jour 1 à 15:35 : quiz de 10 questions, débrief et snapshot des VM.

À retenir

- `kubectl describe node` montre conditions, capacité, ressources réservées et événements : premier réflexe face à un nœud suspect.
- nodeSelector et affinités attirent les pods ; les taints les repoussent ; les tolérances les autorisent.
- Avant toute maintenance : `drain`, puis `uncordon` après. Un PodDisruptionBudget protège les applications critiques.
- Une panne de nœud fait évincer ses pods au bout d'environ 5 minutes : prévoyez au moins 2 réplicas répartis.

Questions de révision

1. Quelle différence entre `cordon` et `drain` ?
2. Pourquoi une tolérance seule ne suffit-elle pas à dédier un nœud à une application ?
3. Vous avez 3 réplicas et 2 workers : quelle règle utiliser pour qu'ils ne finissent pas tous sur le même nœud ?
4. Pourquoi les pods Calico tournent-ils sur le control plane, alors que vos applications n'y vont pas ?

Module 6 — YAML et manifestes

Un manifeste est un fichier YAML qui décrit l'état désiré d'un ou plusieurs objets ; on l'applique avec `kubectl apply -f` et on le versionne dans Git. Maîtriser la syntaxe YAML et l'anatomie commune à tous les objets suffit pour écrire n'importe quelle ressource Kubernetes.

6.1 Pourquoi YAML

L'API Kubernetes échange du JSON. YAML en est un sur-ensemble plus lisible : `kubectl` convertit votre YAML en JSON avant l'envoi. Vous pourriez écrire vos manifestes en JSON, mais personne ne le fait : YAML accepte les commentaires et se lit sans accolades.

6.2 La syntaxe en six règles

1. Une paire s'écrit `cle: valeur`, avec un espace après les deux-points.
2. L'imbrication se fait **par l'indentation, avec des espaces uniquement**. Une tabulation provoque une erreur. Convention : 2 espaces par niveau.
3. Un élément de liste commence par `- ` (tiret, espace).
4. Les dictionnaires et les listes s'imbriquent librement.
5. `#` commence un commentaire.
6. `---` sépare plusieurs documents (plusieurs objets) dans un même fichier.

```
# Un dictionnaire
metadata:
  name: podinfo           # chaîne
  labels:                 # dictionnaire imbriqué
    app: podinfo
# Une liste de dictionnaires : chaque "- " ouvre un nouvel élément
containers:
  - name: web
    image: ghcr.io/stefanprodan/podinfo:6.7.0
    ports:
      - containerPort: 9898
  - name: sidecar
    image: mirror.gcr.io/library/busybox:1.36
# Notation compacte, équivalente
labels: {app: podinfo, tier: web}
args: ["--port", "9898"]
```

Texte sur plusieurs lignes. `|` conserve les retours à la ligne (idéal pour un fichier de configuration ou un script) ; `>` les replie en espaces (pour une longue phrase). Ajouter `-` (`|-`) retire le retour à la ligne final.

```
data:
  nginx.conf: |
    server {
      listen 8080;
    }
  description: >
    Ce texte tient sur plusieurs lignes
    mais sera lu comme une seule.
```

Guillemets. Les guillemets simples prennent tout à la lettre ; les doubles interprètent les séquences d'échappement (`\n`, `\t`). Dans le doute, des guillemets doubles.

6.3 Les pièges de typage

YAML devine le type des valeurs non entre guillemets. Kubernetes, lui, attend des types précis : un label ou une variable d'environnement doit être une chaîne, un port un entier.

Écrit	Compris comme	À écrire si vous voulez du texte
`true`, `false`	Booléen	`"true"`
`yes`, `no`, `on`, `off`	Booléen (YAML 1.1, utilisé par Kubernetes)	`"yes"`
`1.10`	Nombre décimal 1.1	`"1.10"`
`6.7`	Nombre décimal	`"6.7"`
`0644`	Nombre en octal (420) : utile pour `defaultMode`	`"0644"`
`~`, `null`, valeur vide	Absence de valeur	`""` pour une chaîne vide
`*/5 * * * *`	Erreur : `*` débute un alias YAML	`"*/5 * * * *"`
`cle: a: b`	Erreur : second `:` ambigu	`cle: "a: b"`

Inversement, un port écrit `"9898"` entre guillemets est une chaîne, et `containerPort` le refusera. Règle simple : **guillemets pour les valeurs de labels, d'annotations et de variables d'environnement ; pas de guillemets pour les ports, réplicas et tailles numériques**.

6.4 Anatomie d'un objet

Tous les objets partagent la même structure de premier niveau :

```
apiVersion: v1                # groupe/version de l'API (module 2)
kind: Pod                     # type d'objet
metadata:                     # identité et métadonnées
  name: podinfo               # unique par type et par namespace
  namespace: fil-rouge        # facultatif : sinon le namespace courant
  labels:                     # paires clé-valeur pour sélectionner (module
12)
    app: podinfo
  annotations:                # paires clé-valeur pour documenter ou
configurer un outil
    description: "Pod de démonstration"
spec:                         # état désiré : propre à chaque kind
  containers:
    - name: podinfo
      image: ghcr.io/stefanprodan/podinfo:6.7.0
# status:                     # état observé : rempli par Kubernetes, jamais
par vous
```

Règles de nommage. La plupart des noms suivent la norme DNS : minuscules, chiffres et tirets, commençant et finissant par un caractère alphanumérique, 253 caractères au plus (63 pour certains objets comme les Services). `Podinfo` ou `pod_info` sont refusés.

Champs ajoutés par le serveur. Un `kubectl get -o yaml` affiche aussi `uid` (identifiant unique), `resourceVersion` (version de l'objet, pour détecter les modifications concurrentes), `generation`, `creationTimestamp`, et parfois `ownerReferences` (l'objet parent : le ReplicaSet d'un pod) ou `finalizers` (actions à terminer avant suppression). Ne les recopiez pas dans vos fichiers.

6.5 Trouver les bons champs

Inutile d'apprendre les schémas par cœur : la documentation est dans le cluster.

```
kubectl explain pod.spec                          # champs du niveau demandé
kubectl explain pod.spec.containers.resources     # descendre champ par champ
kubectl explain deployment.spec --recursive       # toute l'arborescence
```

Pour partir d'un squelette correct, faites-le générer par `kubectl` sans rien créer :

```
kubectl create deployment web --image=ghcr.io/stefanprodan/podinfo:6.7.0 --replicas=3 --dry-run=client -o yaml > web.yaml
kubectl run test --image=mirror.gcr.io/library/busybox:1.36 --dry-run=client -o yaml
kubectl create configmap conf --from-literal=MODE=prod --dry-run=client -o yaml
kubectl create job sauvegarde --image=mirror.gcr.io/library/busybox:1.36 --dry-run=client -o yaml
kubectl expose deployment web --port=9898 --dry-run=client -o yaml
```

6.6 Impératif ou déclaratif

Approche	Commandes	Avantage	Limite
Commandes impératives	`kubectl run`, `create`, `scale`, `set image`	Rapide pour tester	Aucune trace, non reproductible
Fichiers en mode impératif	`kubectl create -f`, `replace -f`	Trace écrite	Échoue si l'objet existe déjà, ou écrase tout
Fichiers en mode déclaratif	`kubectl apply -f`	Crée ou met à jour, idempotent, versionnable	Demande de la discipline : le fichier fait foi

`kubectl apply` calcule la différence entre votre fichier, la dernière version appliquée (conservée dans une annotation) et l'objet réel, puis n'envoie que les changements. La variante `kubectl apply --server-side` confie ce calcul au serveur et suit quel outil possède quel champ ; elle détecte les conflits entre plusieurs gestionnaires.

Bonne pratique. Les commandes impératives servent à explorer et à dépanner. Toute modification durable passe par le fichier, puis `apply`. Si vous corrigez en urgence avec `kubectl set image` ou `kubectl edit`, reportez la modification dans le fichier ; sinon le prochain `apply` l'annulera. Poussée au bout, cette discipline s'appelle **GitOps** : un outil (Argo CD, Flux) applique en continu le contenu d'un dépôt Git.

6.7 Travailler avec des fichiers

```
kubectl apply -f 30-podinfo.yaml                  # un fichier
kubectl apply -f .                                # tous les fichiers du
dossier, par ordre alphabétique
kubectl apply -f manifests/ -R                    # dossier et sous-dossiers
kubectl diff -f 30-podinfo.yaml                   # ce qu'apply changerait,
sans rien changer
kubectl apply -f 30-podinfo.yaml --dry-run=server # validation complète par
l'API, sans écriture
kubectl delete -f 30-podinfo.yaml                 # supprimer les objets
décrits
```

Préfixez les fichiers par un numéro (`00-namespace.yaml`, `10-redis.yaml`) : `apply -f .` crée alors le namespace avant les objets qui en dépendent.

Kustomize, intégré à `kubectl`, décline une même base pour plusieurs environnements sans copier les fichiers :

```
# overlays/prod/kustomization.yaml
resources:
  - ../../base
namespace: prod
images:
  - name: ghcr.io/stefanprodan/podinfo
    newTag: "6.7.1"
patches:
  - target:
      kind: Deployment
      name: podinfo
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 5
```

```
kubectl kustomize overlays/prod     # afficher le résultat
kubectl apply -k overlays/prod      # l'appliquer
```

6.8 Organiser et contrôler ses manifestes

- Un dossier par application, un fichier par objet ou par groupe cohérent (Deployment et son Service).
- Les manifestes vont dans Git ; **jamais de Secret en clair** dans le dépôt (module 8).
- Contrôlez avant d'appliquer : `yamllint` pour la syntaxe, `kubeconform` pour la conformité au schéma, `kubectl diff` pour l'impact.

- Éditeur : configurez l'indentation à 2 espaces (`set expandtab tabstop=2 shiftwidth=2` dans `~/.vimrc`) ; VS Code avec l'extension YAML de Red Hat complète les champs Kubernetes.

6.9 Messages d'erreur fréquents

Message (extrait)	Cause
`found character that cannot start any token`	Une tabulation dans l'indentation
`mapping values are not allowed in this context`	Indentation incohérente ou `:` en trop dans une valeur
`did not find expected '-' indicator`	Élément de liste mal aligné
`unknown field "spec.contianers"`	Faute de frappe dans un nom de champ
`cannot unmarshal number into ... of type string`	Valeur de label ou d'env sans guillemets
`cannot unmarshal string into ... of type int32`	Port ou nombre entre guillemets
`the namespace from the provided object does not match`	`metadata.namespace` diffère de `-n`
`field is immutable`	Champ non modifiable après création (le `selector` d'un Deployment) : supprimer et recréer

> **TP4 — à faire maintenant** · Jour 2, 09:30 · 35 min
>
> Créer le namespace `fil-rouge`, écrire un premier Pod à la main, puis corriger deux fichiers YAML cassés à l'aide du tableau 6.9.
>
> **Point de contrôle :** le contexte courant pointe sur `fil-rouge` ; les fichiers corrigés passent `kubectl apply --dry-run=server`.

À retenir

- Indentation par espaces, jamais de tabulation ; `-` pour les listes ; `---` entre les objets.
- Mettez entre guillemets les valeurs de labels, annotations et variables d'environnement.
- Tout objet = `apiVersion`, `kind`, `metadata`, `spec` ; `status` est écrit par Kubernetes.

- `kubectl explain` et `--dry-run=client -o yaml` évitent d'apprendre les schémas par cœur.
- Le fichier fait foi : modifiez-le, puis `kubectl diff` et `kubectl apply`.

Questions de révision

1. Pourquoi `version: 1.10` pose-t-il problème dans les labels d'un pod ?
2. Quelle différence entre `kubectl create -f` et `kubectl apply -f` quand l'objet existe déjà ?
3. Comment obtenir un squelette de CronJob correct sans le rédiger ?
4. Vous avez corrigé une image avec `kubectl set image` : que risquez-vous au prochain `apply` ?

Module 7 — Pods, Deployments, Services

On ne déploie presque jamais un pod seul : un Deployment maintient le bon nombre de pods identiques et gère leurs versions, et un Service leur donne une adresse et un nom stables. Ces trois objets, rangés dans un namespace, forment la base de toute application sur Kubernetes.

7.1 Le Pod

Le pod est la plus petite unité que Kubernetes déploie : un ou plusieurs conteneurs qui partagent :

- **le réseau** : une seule adresse IP pour le pod ; ses conteneurs se joignent par `localhost` et ne peuvent pas écouter sur le même port ;
- **les volumes** déclarés dans le pod ;
- **un cycle de vie** : ils sont placés ensemble sur le même nœud et supprimés ensemble.

Un pod est **éphémère** : s'il meurt ou si son nœud tombe, il n'est pas réparé mais remplacé par un nouveau pod, avec un autre nom et une autre IP. C'est pourquoi on le confie toujours à un contrôleur.

Plusieurs conteneurs dans un pod. La règle : un conteneur principal par pod. On en ajoute d'autres seulement s'ils doivent vivre avec lui :

- un **conteneur d'initialisation** (`initContainers`) s'exécute jusqu'au bout avant le démarrage des autres : attendre une base de données, préparer un fichier ;
- un **sidecar** accompagne le conteneur principal pendant toute sa vie : agent de logs, proxy. Kubernetes le déclare nativement comme un conteneur d'initialisation avec `restartPolicy: Always`, ce qui garantit qu'il démarre avant et s'arrête après

l'application.

```
apiVersion: v1
kind: Pod
metadata:
  name: exemple
  labels:
    app: exemple
spec:
  initContainers:
    - name: attendre-redis
      image: mirror.gcr.io/library/busybox:1.36
      command: ["sh", "-c", "until nc -z redis 6379; do sleep 2; done"]
  containers:
    - name: app
      image: ghcr.io/stefanprodan/podinfo:6.7.0
      ports:
        - containerPort: 9898
          name: http
  restartPolicy: Always          # Always (défaut), OnFailure ou Never
```

Cycle de vie. Un pod traverse des phases : `Pending` (accepté, pas encore démarré : placement, téléchargement d'image), `Running`, puis `Succeeded` ou `Failed` pour un pod qui se termine. Chaque conteneur a son propre état : `Waiting` (avec une raison : `ContainerCreating`, `ImagePullBackOff`, `CrashLoopBackOff`…), `Running` ou `Terminated`.

Arrêt propre. À la suppression d'un pod, le kubelet envoie le signal `SIGTERM` au processus principal, attend la fin du délai de grâce (30 secondes par défaut, `terminationGracePeriodSeconds`), puis envoie `SIGKILL`. Une application bien écrite intercepte `SIGTERM` pour finir ses requêtes en cours. Un crochet `preStop` permet d'exécuter une commande avant l'envoi du signal.

7.2 Le ReplicaSet

Un ReplicaSet garantit qu'un nombre donné de pods correspondant à un `selector` existe en permanence ; il crée les pods à partir d'un modèle (`template`). Vous ne l'écrirez jamais vous-même : le Deployment le gère pour vous.

7.3 Le Deployment

Le Deployment est l'objet standard pour une application sans état. Il crée un ReplicaSet par version de votre modèle de pod, ce qui permet les mises à jour progressives et les retours arrière (module 9).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
  labels:
    app: podinfo
spec:
  replicas: 3                    # nombre de pods voulus
  selector:                      # quels pods ce Deployment gère (non
modifiable ensuite)
    matchLabels:
      app: podinfo
  template:                      # modèle des pods à créer
    metadata:
      labels:
        app: podinfo             # doit correspondre au selector
    spec:
      containers:
        - name: podinfo
          image: ghcr.io/stefanprodan/podinfo:6.7.0
          ports:
            - containerPort: 9898
```

La chaîne de propriété se lit dans les noms : le Deployment `podinfo` possède le ReplicaSet `podinfo-5d9c7b8f6` (suffixe = empreinte du modèle), qui possède les pods `podinfo-5d9c7b8f6-x2x7k`, etc.

```
kubectl get deploy,rs,pods -l app=podinfo
kubectl scale deployment podinfo --replicas=5
kubectl describe deployment podinfo      # stratégie, conditions, événements
```

Champs utiles : `revisionHistoryLimit` (anciens ReplicaSets conservés, 10 par défaut), `minReadySeconds` (durée pendant laquelle un pod doit rester prêt pour compter comme disponible), `progressDeadlineSeconds` (délai au-delà duquel une mise à jour bloquée est signalée en échec, 600 s par défaut).

7.4 Les namespaces

Un namespace est un espace de noms qui regroupe des objets : deux Services peuvent s'appeler `redis` dans deux namespaces différents. C'est aussi l'unité à laquelle on attache des quotas (module 11), des droits RBAC et des politiques de sécurité (module 15).

Namespace	Contenu
`default`	Objets créés sans namespace précisé
`kube-system`	Composants du cluster (CoreDNS, kube-proxy, pods statiques)
`kube-public`	Informations lisibles par tous, y compris sans authentification
`kube-node-lease`	Battements de cœur des nœuds

```
kubectl get namespaces
kubectl create namespace fil-rouge
kubectl config set-context --current --namespace=fil-rouge   # namespace par défaut de vos commandes
kubectl get pods -A                                          # tous les namespaces
```

Attention : un namespace **n'isole pas le réseau**. Par défaut, un pod de `equipe-a` joint un pod de `equipe-b` ; il faut des NetworkPolicies pour l'empêcher. Et supprimer un namespace supprime tout ce qu'il contient.

7.5 Les Services

Les pods naissent et meurent, leurs adresses changent. Un Service offre en permanence :

- une **adresse IP virtuelle** stable (la ClusterIP), prise dans 10.96.0.0/12 ;
- un **nom DNS** stable ;
- une **répartition de charge** entre tous les pods prêts qui correspondent à son `selector`.

Le contrôleur EndpointSlice tient à jour la liste des adresses des pods derrière chaque Service ; kube-proxy en déduit des règles sur chaque nœud, qui redirigent le trafic de la ClusterIP vers l'un des pods.

Type	Accessible depuis	Usage
`ClusterIP` (défaut)	L'intérieur du cluster	Communication entre composants (podinfo vers Redis)
`NodePort`	L'extérieur, sur un port 30000-32767 de chaque nœud	Tests, labos, répartiteur externe manuel

Type	Accessible depuis	Usage
`LoadBalancer`	L'extérieur, via un répartiteur fourni par le Cloud	Exposition en production sur un service managé
`ExternalName`	L'intérieur : alias DNS vers un nom externe	Pointer vers une base hébergée hors du cluster
Headless (`clusterIP: None`)	L'intérieur : le DNS renvoie directement les IP des pods	Bases de données en StatefulSet (module 16)

Quatre numéros de port à ne pas confondre :

Champ	Où	Signification
`containerPort`	Pod	Port sur lequel l'application écoute (informatif)
`port`	Service	Port de la ClusterIP, celui qu'appellent les clients
`targetPort`	Service	Port du pod vers lequel le trafic est renvoyé (nombre ou nom de port)
`nodePort`	Service NodePort	Port ouvert sur chaque nœud

```
apiVersion: v1
kind: Service
metadata:
  name: podinfo
spec:
  type: NodePort
  selector:
    app: podinfo          # tous les pods prêts portant ce label
  ports:
    - name: http
      port: 9898          # port du Service
      targetPort: http    # port nommé du conteneur
      nodePort: 30080     # facultatif : sinon choisi au hasard dans la plage
```

```
kubectl get svc podinfo
kubectl get endpointslices -l kubernetes.io/service-name=podinfo   # les pods derrière le Service
kubectl expose deployment podinfo --port=9898                      # création impérative
```

Un Service sans aucun endpoint est le symptôme classique d'un `selector` qui ne correspond à aucun label de pod, ou de pods qui ne sont pas prêts (module 9).

7.6 Le DNS interne

CoreDNS crée un nom pour chaque Service : `<service>.<namespace>.svc.cluster.local`. Les pods reçoivent un fichier `/etc/resolv.conf` avec des domaines de recherche, qui permettent les formes courtes :

Depuis un pod de…	Nom utilisable pour joindre `redis` du namespace `fil-rouge`
`fil-rouge`	`redis`
un autre namespace	`redis.fil-rouge`
partout	`redis.fil-rouge.svc.cluster.local`

```
kubectl run dns --image=mirror.gcr.io/library/busybox:1.36 --rm -it --
restart=Never -- sh
# dans le pod :
cat /etc/resolv.conf
nslookup redis.fil-rouge
wget -qO- http://podinfo.fil-rouge:9898
```

Pour un Service headless, le nom renvoie la liste des IP des pods au lieu d'une ClusterIP.

7.7 Exposer une application en HTTP

Un NodePort par application ne passe pas à l'échelle. En production, une seule entrée HTTP route le trafic vers les Services selon le nom d'hôte et le chemin :

- **Ingress** : l'API historique, simple et très répandue. Elle ne fait rien seule : il faut un contrôleur Ingress. Son API n'évolue plus.
- **Gateway API** : son successeur, stable depuis 2023, plus expressif (répartition pondérée, en-têtes, plusieurs protocoles) et qui sépare les rôles : l'équipe plateforme définit la `Gateway`, les équipes applicatives leurs `HTTPRoute`. C'est le choix recommandé pour un nouveau cluster.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: podinfo
  namespace: fil-rouge
spec:
  parentRefs:
    - name: passerelle-principale      # Gateway définie par l'équipe
plateforme
      namespace: infra
  hostnames: ["podinfo.exemple.fr"]
  rules:
    - backendRefs:
        - name: podinfo
          port: 9898
```

Pour un accès ponctuel de dépannage, sans rien exposer : `kubectl port-forward svc/podinfo 9898:9898`, puis `http://localhost:9898` depuis la machine où tourne `kubectl`.

7.8 L'application fil rouge

Au TP5, vous déployez dans le namespace `fil-rouge` :

- un Deployment `redis` (1 réplica) et un Service `redis` de type ClusterIP sur le port 6379 ;
- un Deployment `podinfo` (3 réplicas) lancé avec l'option `--cache-server=tcp://redis:6379` : il joint Redis par son nom DNS court ;
- un Service `podinfo` de type NodePort sur le port 30080, pour l'atteindre depuis l'extérieur du cluster (dans nos TP, avec curl depuis k8s-cp).

Le chemin d'une requête : client HTTP (curl ou navigateur) → port 30080 d'un nœud → règles kube-proxy → l'un des 3 pods podinfo → Service `redis` → pod Redis.

À retenir

- Un pod = un ou plusieurs conteneurs partageant IP et volumes ; il est éphémère et toujours géré par un contrôleur.
- Deployment → ReplicaSet → Pods : les labels du modèle doivent correspondre au `selector`.
- Un Service donne une IP, un nom DNS et une répartition de charge stables devant des pods qui changent.
- `port` est celui du Service, `targetPort` celui du conteneur, `nodePort` celui du nœud.

- Un namespace range et délègue, mais n'isole pas le réseau.

Questions de révision

1. Deux conteneurs d'un même pod peuvent-ils écouter tous deux sur le port 8080 ? Pourquoi ?
2. Que se passe-t-il si le `selector` d'un Service ne correspond à aucun pod ?
3. Quel nom utiliser pour joindre le Service `redis` du namespace `fil-rouge` depuis le namespace `equipe-a` ?
4. Pourquoi un pod créé seul, sans Deployment, est-il déconseillé ?
5. Quelle différence entre un Service NodePort et un Service LoadBalancer ?

Module 8 — Configuration : ConfigMaps et Secrets

La configuration se sépare de l'image : la même image tourne en recette et en production, seuls changent les ConfigMaps (paramètres) et les Secrets (mots de passe, clés). Un Secret n'est qu'encodé en base64 : sa protection repose sur les droits d'accès, le chiffrement d'etcd et, idéalement, un coffre-fort externe.

8.1 Pourquoi séparer configuration et image

Reconstruire une image pour changer une adresse de base de données est lent et risqué : l'image testée n'est plus celle qu'on déploie. Le principe, popularisé par la méthode des « 12 facteurs », est de construire une image unique et de lui injecter sa configuration au démarrage, sous forme de variables d'environnement ou de fichiers.

8.2 Les ConfigMaps

Une ConfigMap stocke des paires clé-valeur non sensibles, jusqu'à 1 Mio au total. Une valeur peut être courte (`MODE=prod`) ou contenir un fichier entier.

```
kubectl create configmap app-config --from-literal=MODE=prod --from-
literal=NIVEAU_LOG=info
kubectl create configmap nginx-conf --from-file=nginx.conf       # clé = nom
du fichier
kubectl create configmap env-conf --from-env-file=app.env        # une clé
par ligne CLE=valeur
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: podinfo-config
data:
  PODINFO_UI_MESSAGE: "Bienvenue à l'INSEE"
  PODINFO_UI_COLOR: "#2b5797"
  config.yaml: |
    niveau: info
    cache:
      ttl: 300
```

Trois façons de la consommer :

```
spec:
  containers:
    - name: app
      image: ghcr.io/stefanprodan/podinfo:6.7.0
      # 1. Une clé précise dans une variable
      env:
        - name: MESSAGE
          valueFrom:
            configMapKeyRef:
              name: podinfo-config
              key: PODINFO_UI_MESSAGE
      # 2. Toutes les clés comme variables
      envFrom:
        - configMapRef:
            name: podinfo-config
      # 3. Les clés comme fichiers dans un répertoire
      volumeMounts:
        - name: config
          mountPath: /etc/podinfo
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: podinfo-config
        items:                      # facultatif : seulement certaines clés
          - key: config.yaml
            path: config.yaml
```

Une variable peut aussi alimenter les arguments du conteneur avec la syntaxe `$(NOM)` : `args: ["--message=$(MESSAGE)"]`.

Que se passe-t-il quand on modifie une ConfigMap ?

Consommation	Prise en compte de la modification
Variable d'environnement (`env` , `envFrom`)	Jamais : seulement au prochain démarrage du conteneur
Volume	Les fichiers sont mis à jour après un délai (de l'ordre de la minute) ; l'application doit relire le fichier
Volume monté avec `subPath`	Jamais

Pour appliquer une nouvelle configuration, la méthode fiable est de redémarrer les pods de façon progressive : `kubectl rollout restart deployment podinfo` . Les outils comme Helm et Kustomize automatisent ce redémarrage en ajoutant au modèle de pod une annotation qui change avec le contenu de la configuration.

Une ConfigMap peut être déclarée `immutable: true` : elle ne peut plus être modifiée, seulement remplacée par une nouvelle (`podinfo-config-v2`). Cela évite les modifications accidentelles et allège la charge de l'API server.

8.3 Les Secrets

Un Secret ressemble à une ConfigMap, mais il est destiné aux données sensibles et bénéficie de quelques précautions : il est monté en mémoire (tmpfs) et non sur disque, n'est envoyé qu'aux nœuds qui en ont besoin, et peut être chiffré dans etcd.

Type	Usage
`Opaque`	Données libres (défaut)
`kubernetes.io/tls`	Certificat et clé privée (`tls.crt` , `tls.key`)
`kubernetes.io/dockerconfigjson`	Identifiants d'un registre d'images privé
`kubernetes.io/basic-auth` , `kubernetes.io/ssh-auth`	Identifiants structurés

```
kubectl create secret generic podinfo-secret --from-literal=API_TOKEN=s3cr3t-
insee
kubectl create secret tls podinfo-tls --cert=tls.crt --key=tls.key
kubectl create secret docker-registry registre-prive --docker-
server=registry.exemple.fr --docker-username=robot --docker-password='...'
```

Dans un fichier YAML, `data` attend des valeurs encodées en base64 ; `stringData` accepte du texte clair, que l'API encode à l'enregistrement :

```
apiVersion: v1
kind: Secret
metadata:
  name: podinfo-secret
type: Opaque
stringData:
  API_TOKEN: s3cr3t-insee
```

On consomme un Secret comme une ConfigMap (`secretKeyRef`, `secretRef`, volume `secret`). Pour télécharger depuis un registre privé, on le référence dans le pod :

```
spec:
  imagePullSecrets:
    - name: registre-prive
```

8.4 Ce qu'un Secret protège, et ce qu'il ne protège pas

base64 n'est pas un chiffrement. Quiconque lit l'objet lit le secret :

```
kubectl get secret podinfo-secret -o jsonpath='{.data.API_TOKEN}' | base64 -d
```

Risques à connaître :

- par défaut, les Secrets sont stockés **en clair dans etcd** : une sauvegarde d'etcd volée les révèle tous ;
- toute personne qui peut créer un pod dans un namespace peut y monter n'importe quel Secret de ce namespace, et donc le lire ;
- un Secret passé en variable d'environnement peut fuiter dans les logs, les rapports d'erreur ou `kubectl describe` d'outils mal configurés.

Mesures de protection, par ordre de priorité :

1. **RBAC** : limiter `get`, `list` et `watch` sur les Secrets aux seules identités qui en ont besoin (module 15). Le droit `list` suffit à lire tous les Secrets d'un namespace.
2. **Chiffrement au repos** : configurer l'API server pour chiffrer les Secrets dans etcd, avec une clé locale ou, mieux, un service de gestion de clés (KMS).
3. **Coffre-fort externe** : garder la valeur de référence dans HashiCorp Vault ou le gestionnaire de secrets d'un Cloud, et la synchroniser dans le cluster avec External Secrets Operator ou le pilote Secrets Store CSI.
4. **Rien en clair dans Git** : chiffrer les Secrets versionnés avec Sealed Secrets ou SOPS.
5. **Préférer les fichiers aux variables** pour les secrets les plus sensibles.

Exemple de chiffrement au repos, fichier référencé par l'option `--encryption-provider-config` dans `/etc/kubernetes/manifests/kube-apiserver.yaml` :

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources: ["secrets"]
    providers:
      - aescbc:
          keys:
            - name: cle1
              secret: <32 octets aléatoires encodés en base64>
      - identity: {}        # permet encore de lire les anciens Secrets non
chiffrés
```

8.5 Injecter des informations sur le pod lui-même

L'**API Downward** expose au conteneur ses propres métadonnées, sans qu'il interroge l'API server :

```
env:
  - name: NOM_POD
    valueFrom:
      fieldRef:
        fieldPath: metadata.name
  - name: NAMESPACE
    valueFrom:
      fieldRef:
        fieldPath: metadata.namespace
  - name: NOEUD
    valueFrom:
      fieldRef:
        fieldPath: spec.nodeName
  - name: LIMITE_MEMOIRE
    valueFrom:
      resourceFieldRef:
        resource: limits.memory
```

8.6 Dans l'application fil rouge

Au TP5, podinfo lit son message d'accueil et sa couleur dans la ConfigMap `podinfo-config`, et un jeton fictif dans le Secret `podinfo-secret`, les deux via `envFrom`. Vous constatez ensuite qu'une modification de la ConfigMap reste invisible jusqu'à un `kubectl rollout restart`.

> **TP5 — à faire maintenant** · Jour 2, 10:30 · 70 min
>
> Déployer l'application fil rouge : Redis et podinfo, leurs Services, une ConfigMap et un Secret (sections 7.3 à 7.8 et 8.2 à 8.6).
>
> **Point de contrôle :** 3 pods podinfo et 1 pod Redis en `Running`, interface affichée sur le port 30080, écriture puis lecture dans le cache.

À retenir

- ConfigMap pour les paramètres, Secret pour les données sensibles ; une seule image pour tous les environnements.
- Variables d'environnement : lues au démarrage seulement. Volumes : mis à jour, sauf avec `subPath`.
- Un Secret est encodé, pas chiffré : protégez-le par RBAC, chiffrement d'etcd et coffre-fort externe.
- Ne versionnez jamais un Secret en clair.

Questions de révision

1. Vous modifiez une ConfigMap consommée par `envFrom` : que voient les pods en cours ? Que faire ?
2. Pourquoi le droit `list` sur les Secrets est-il aussi sensible que le droit `get` ?
3. Quelle différence entre les champs `data` et `stringData` d'un Secret ?
4. Citez deux façons d'éviter de stocker des Secrets en clair dans Git.

Module 9 — Cycle de vie des applications

Un Deployment met à jour ses pods progressivement et sait revenir à la version précédente ; les probes lui disent quand un pod est prêt ou bloqué. D'autres contrôleurs couvrent les autres formes de travail : Job pour une tâche qui se termine, CronJob pour une tâche planifiée, DaemonSet pour un pod par nœud.

9.1 Les stratégies de mise à jour

Une mise à jour (un *rollout*) se déclenche dès que le **modèle de pod** change : image, variable d'environnement, label du modèle, ressources. Changer le nombre de réplicas ne déclenche pas de rollout.

RollingUpdate (défaut) : le Deployment crée un nouveau ReplicaSet, le fait grandir et réduit l'ancien pas à pas. Deux paramètres règlent le rythme :

- `maxSurge` : combien de pods **en plus** du nombre voulu sont tolérés pendant la mise à jour (25 % par défaut, arrondi au supérieur) ;
- `maxUnavailable` : combien de pods peuvent **manquer** (25 % par défaut, arrondi à l'inférieur).

Avec 4 réplicas et les valeurs par défaut : au plus 5 pods existent, au moins 3 sont disponibles à tout instant.

Réglage	Effet	Condition
`maxSurge: 1`, `maxUnavailable: 0`	Jamais moins de pods que voulu : aucune perte de capacité	Le cluster doit avoir de la place pour un pod de plus
`maxSurge: 0`, `maxUnavailable: 1`	Aucune ressource supplémentaire	Capacité réduite d'un pod pendant la mise à jour
Valeurs par défaut (25 %)	Compromis vitesse et capacité	—

Recreate : tous les anciens pods sont supprimés avant la création des nouveaux. Il y a donc une coupure. On la choisit quand deux versions ne peuvent pas coexister (schéma de base de données incompatible) ou quand un volume ne peut être monté que par un seul pod à la fois (module 16).

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

9.2 Suivre et piloter une mise à jour

```
kubectl set image deployment/podinfo
podinfo=ghcr.io/stefanprodan/podinfo:6.7.1
kubectl annotate deployment/podinfo kubernetes.io/change-cause="passage en
6.7.1" --overwrite
kubectl rollout status deployment/podinfo           # suit la progression
jusqu'au bout
kubectl rollout history deployment/podinfo          # révisions et leur cause
kubectl rollout history deployment/podinfo --revision=2   # détail d'une
révision
kubectl rollout undo deployment/podinfo             # revenir à la révision
précédente
kubectl rollout undo deployment/podinfo --to-revision=1
kubectl rollout pause deployment/podinfo            # suspendre (pour grouper
plusieurs changements)
kubectl rollout resume deployment/podinfo
kubectl rollout restart deployment/podinfo          # redémarrer tous les
pods progressivement
```

Chaque révision correspond à un ReplicaSet conservé (10 par défaut) : un retour arrière ne fait que réactiver un ancien ReplicaSet. Le texte de l'annotation `kubernetes.io/change-cause` apparaît dans la colonne CHANGE-CAUSE de l'historique.

Mise à jour bloquée. Si les nouveaux pods ne deviennent jamais prêts (image introuvable, application qui plante), le rollout s'arrête : grâce à `maxUnavailable`, les anciens pods continuent de servir. Au bout de `progressDeadlineSeconds` (600 s par défaut), le Deployment affiche la condition `Progressing=False` avec la raison `ProgressDeadlineExceeded`. Kubernetes ne revient **pas** en arrière tout seul : c'est à vous, à votre chaîne de déploiement ou à un outil comme Argo Rollouts de décider.

9.3 Les probes

Une probe est un test que le kubelet exécute régulièrement sur un conteneur. Trois types répondent à trois questions différentes :

Probe	Question	En cas d'échec
`startupProbe`	L'application a-t-elle fini de démarrer ?	Les autres probes attendent ; au-delà du seuil, le conteneur est redémarré
`readinessProbe`	Peut-elle recevoir du trafic maintenant ?	Le pod est retiré des Services, sans être redémarré

Probe	Question	En cas d'échec
`livenessProbe`	Est-elle bloquée sans espoir de rétablissement ?	Le conteneur est redémarré

Quatre mécanismes : `httpGet` (succès si code HTTP entre 200 et 399), `tcpSocket` (succès si le port accepte la connexion), `exec` (succès si la commande renvoie 0), `grpc` (protocole de santé gRPC).

Paramètre	Rôle	Défaut
`initialDelaySeconds`	Attente avant le premier test	0
`periodSeconds`	Intervalle entre deux tests	10
`timeoutSeconds`	Délai de réponse maximal	1
`failureThreshold`	Échecs consécutifs avant de conclure à l'échec	3
`successThreshold`	Succès consécutifs pour conclure au succès	1

```
containers:
  - name: podinfo
    image: ghcr.io/stefanprodan/podinfo:6.7.1
    startupProbe:                # jusqu'à 30 x 2 s = 60 s pour démarrer
      httpGet:
        path: /healthz
        port: 9898
      periodSeconds: 2
      failureThreshold: 30
    readinessProbe:
      httpGet:
        path: /readyz
        port: 9898
      periodSeconds: 5
    livenessProbe:
      httpGet:
        path: /healthz
        port: 9898
      periodSeconds: 10
      failureThreshold: 3
```

Bonnes pratiques.

- Une liveness ne teste **que le processus lui-même**, jamais une dépendance. Si elle vérifie la base de données, une panne de la base fait redémarrer tous les pods en boucle, ce qui aggrave l'incident.
- La readiness peut, elle, tenir compte de ce dont l'application a besoin pour servir (cache chargé, connexion établie).
- Pour une application lente à démarrer, utilisez une `startupProbe` plutôt qu'un long `initialDelaySeconds` sur la liveness.
- Sans readiness, un pod reçoit du trafic dès que son conteneur démarre, avant que l'application soit prête : erreurs garanties pendant chaque mise à jour.

Redémarrages et CrashLoopBackOff. Quand un conteneur s'arrête en erreur, le kubelet le relance avec un délai croissant (10 s, 20 s, 40 s…, plafonné à 5 minutes). L'état `CrashLoopBackOff` signifie « en attente avant la prochaine tentative ». La colonne RESTARTS de `kubectl get pods` compte les redémarrages.

9.4 Mises à jour sans erreur

Deux mécanismes combinés garantissent qu'aucune requête n'échoue pendant un rollout :

1. **À l'arrivée**, la readiness probe retient le trafic tant que le nouveau pod n'est pas prêt.
2. **Au départ**, l'application doit finir proprement : quand un pod est supprimé, son retrait des Services et l'envoi de `SIGTERM` se font en parallèle. Quelques requêtes peuvent encore arriver juste après le signal. Une courte pause dans un crochet `preStop` laisse aux règles réseau le temps de se mettre à jour :

```
lifecycle:
  preStop:
    exec:
      command: ["sh", "-c", "sleep 5"]   # les versions récentes offrent
aussi une action native sleep
```

9.5 Les Jobs

Un Job exécute un ou plusieurs pods **jusqu'à leur terme** : traitement par lots, calcul, migration, sauvegarde. Un Job réussi n'est pas relancé.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: calcul-indices
spec:
  completions: 5               # 5 exécutions réussies au total
  parallelism: 2               # 2 pods en même temps
  backoffLimit: 4              # échecs tolérés avant d'abandonner (6 par
défaut)
  activeDeadlineSeconds: 3600  # durée maximale du Job
  ttlSecondsAfterFinished: 86400  # suppression automatique 24 h après la fin
  template:
    spec:
      restartPolicy: Never     # Never ou OnFailure, jamais Always pour un
Job
      containers:
        - name: calcul
          image: mirror.gcr.io/library/python:3.12-alpine
          command: ["python", "-c", "print('calcul terminé')"]
```

```
kubectl get jobs
kubectl get pods -l job-name=calcul-indices
kubectl logs job/calcul-indices
```

Pour répartir un travail en morceaux, un Job indexé (`completionMode: Indexed`) donne à chaque pod un numéro distinct dans la variable `JOB_COMPLETION_INDEX` .

9.6 Les CronJobs

Un CronJob crée un Job selon un calendrier, au format cron classique en 5 champs :

Champ	Valeurs	Exemple `30 2 * * 1-5`
Minute	0-59	30
Heure	0-23	2
Jour du mois	1-31	tous
Mois	1-12	tous
Jour de la semaine	0-6 (0 = dimanche)	du lundi au vendredi

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: export-nocturne
spec:
  schedule: "30 2 * * 1-5"        # guillemets indispensables
  timeZone: "Europe/Paris"        # sinon le fuseau du control plane
  concurrencyPolicy: Forbid       # Allow, Forbid ou Replace si l'exécution
précédente tourne encore
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: export
              image: mirror.gcr.io/library/busybox:1.36
              command: ["sh", "-c", "date; echo export terminé"]
```

```
kubectl get cronjobs
kubectl create job test-manuel --from=cronjob/export-nocturne   # déclencher
tout de suite
kubectl patch cronjob export-nocturne -p '{"spec":{"suspend":true}}'   #
suspendre
```

9.7 Les DaemonSets

Un DaemonSet fait tourner **un pod sur chaque nœud** (ou sur chaque nœud sélectionné par un `nodeSelector`). Quand un nœud rejoint le cluster, il reçoit automatiquement son pod. Usages typiques : plugin réseau (`calico-node`), `kube-proxy`, agent de collecte de logs, exportateur de métriques système.

```
kubectl get daemonsets -A
kubectl get pods -n calico-system -o wide -l k8s-app=calico-node   # un par
nœud
```

9.8 Quel contrôleur pour quel besoin

Besoin	Contrôleur
Application web ou API sans état	Deployment

Besoin	Contrôleur
Base de données, service qui a besoin d'une identité et d'un stockage stables	StatefulSet (module 16)
Un pod sur chaque nœud	DaemonSet
Tâche qui se termine	Job
Tâche planifiée	CronJob

> **TP6 — à faire maintenant** · Jour 2, 13:00 · 70 min
>
> Mettre à jour podinfo en 6.7.1, provoquer une mise à jour ratée et revenir en arrière, puis ajouter readiness et liveness probes (sections 9.1 à 9.4).
>
> **Point de contrôle :** podinfo en 6.7.1, 3/3 `READY`, au moins 3 révisions dans `rollout history`.

À retenir

- Toute modification du modèle de pod déclenche un rollout ; `kubectl rollout undo` réactive l'ancien ReplicaSet.
- `maxSurge` et `maxUnavailable` règlent le compromis entre vitesse, capacité et ressources.
- Readiness retire du trafic, liveness redémarre, startup protège les démarrages lents. Une liveness ne teste jamais une dépendance.
- Kubernetes ne revient pas en arrière tout seul après une mise à jour ratée.
- Job pour une tâche finie, CronJob pour une tâche planifiée (schéma entre guillemets), DaemonSet pour un pod par nœud.

Questions de révision

1. Avec 10 réplicas et les valeurs par défaut, combien de pods au maximum pendant un rollout ? Combien disponibles au minimum ?
2. Pourquoi l'application répond-elle toujours pendant un rollout vers une image introuvable ?
3. Votre liveness interroge la base de données. Que se passe-t-il si la base tombe ?
4. Comment exécuter immédiatement un CronJob pour le tester ?

Module 10 — Surveiller et diagnostiquer

Presque toutes les pannes se résolvent avec quatre commandes, dans cet ordre : `get` pour voir l'état, `describe` pour lire les événements, `logs` pour entendre l'application, `events` pour la chronologie. Ce module donne la méthode, la signification de chaque état et les outils pour aller plus loin.

10.1 La méthode

1. **`kubectl get`** : quel objet est en anomalie ? Lisez les colonnes READY, STATUS et RESTARTS.
2. **`kubectl describe`** : lisez la section **Events** en bas de la sortie. Elle donne la cause dans la majorité des cas.
3. **`kubectl logs`** : ce que l'application a écrit, y compris avant son dernier plantage (`--previous`).
4. **`kubectl events`** : la chronologie de tout le namespace, pour les problèmes qui touchent plusieurs objets.
5. Si besoin : `exec`, `port-forward` ou `debug` pour observer de l'intérieur.

Pour un problème d'accès, descendez la chaîne : Service → EndpointSlice (le Service a-t-il des pods ?) → pods (sont-ils prêts ?) → conteneur (répond-il sur le bon port ?) → nœud.

10.2 Lire l'état d'un pod

```
NAME                       READY   STATUS             RESTARTS      AGE
podinfo-6c9f8d7b5-4kx2p    1/1     Running            0             2h
podinfo-6c9f8d7b5-9mz7q    0/1     Running            0             5m
panne-7f8b9c6d4-l2k8w      0/1     CrashLoopBackOff   6 (2m ago)    10m
```

READY compte les conteneurs prêts sur le total ; un pod `Running` mais `0/1` ne reçoit pas de trafic.

État	Signification	Causes fréquentes	Première commande
`Pending`	Pas encore placé sur un nœud	Ressources insuffisantes, taint non tolérée, nodeSelector impossible, volume non lié, quota	`describe pod` : FailedScheduling

État	Signification	Causes fréquentes	Première commande
`ContainerCreating`	Placé, conteneur en préparation	Téléchargement d'image long, volume impossible à monter, problème CNI	`describe pod`
`ErrImagePull`, `ImagePullBackOff`	Image impossible à télécharger	Tag inexistant, registre privé sans identifiants, limite de téléchargement, pas d'accès Internet	`describe pod` : message du registre
`CrashLoopBackOff`	Le conteneur démarre puis s'arrête en boucle	Erreur applicative, configuration manquante, liveness trop stricte	`logs --previous`
`CreateContainerConfigError`	Configuration du conteneur impossible à construire	ConfigMap ou Secret référencé inexistant	`describe pod`
`RunContainerError`, `StartError`	Le runtime n'a pas pu lancer la commande	Exécutable introuvable, droits	`describe pod`
`OOMKilled` (dans Last State)	Tué pour dépassement de sa limite mémoire	Limite trop basse ou fuite mémoire	`describe pod`, `top pod`
`Running` mais `0/1`	Readiness en échec	Application pas prête, mauvais chemin ou port de probe	`describe pod` : Readiness probe failed
`Terminating` qui dure	Suppression bloquée	Nœud injoignable, finalizer non levé	`describe pod`, état du nœud

État	Signification	Causes fréquentes	Première commande
`Evicted`	Expulsé par le kubelet	Pression mémoire ou disque sur le nœud	`describe node`
`Completed`	Terminé avec succès	Normal pour un Job	—

10.3 Les logs

Kubernetes récupère ce que les conteneurs écrivent sur leur sortie standard et leur sortie d'erreur. Une application conteneurisée doit donc journaliser sur la console, pas dans un fichier.

```
kubectl logs podinfo-6c9f8d7b5-4kx2p             # logs d'un pod
kubectl logs -f deploy/podinfo                    # suivre en direct (un pod
du Deployment)
kubectl logs -l app=podinfo --prefix              # tous les pods d'un label,
avec leur nom
kubectl logs <pod> --previous                     # instance précédente,
avant le plantage
kubectl logs <pod> -c <conteneur>                 # pod à plusieurs
conteneurs
kubectl logs <pod> --since=15m --tail=100 --timestamps
```

Les logs sont stockés sur le nœud (`/var/log/pods/`), avec une rotation par taille. Ils disparaissent avec le pod. En production, un agent (Fluent Bit, par exemple) les envoie vers un système central (Loki, Elasticsearch) pour les conserver et les rechercher.

10.4 Les événements

Les événements racontent ce que les composants ont fait : placement, téléchargement, démarrage, échec de probe, éviction. Ils sont conservés une heure par défaut.

```
kubectl events                                    # namespace courant,
chronologique
kubectl events --types=Warning                    # seulement les
avertissements
kubectl events --for=pod/podinfo-6c9f8d7b5-4kx2p  # un objet précis
kubectl events -A -w                              # tout le cluster, en
direct
kubectl get events --sort-by=.lastTimestamp       # forme ancienne,
équivalente
```

10.5 Observer de l'intérieur

```
kubectl exec -it <pod> -- sh                          # shell dans le conteneur
kubectl exec <pod> -- env                             # variables réellement
reçues
kubectl cp <pod>:/etc/config/app.yaml ./app.yaml      # copier un fichier
kubectl port-forward pod/<pod> 9898:9898              # joindre le pod depuis
votre machine
```

Beaucoup d'images de production ne contiennent ni shell ni outils (images *distroless*). `kubectl debug` ajoute alors un **conteneur éphémère** équipé dans le pod, qui partage son réseau et peut voir ses processus :

```
kubectl debug -it <pod> --image=mirror.gcr.io/library/busybox:1.36 --
target=podinfo
kubectl debug node/k8s-w1 -it --image=mirror.gcr.io/library/busybox:1.36   #
shell sur le nœud, son disque sous /host
kubectl debug <pod> -it --copy-to=copie --container=podinfo -- sh          #
copie du pod avec une autre commande
```

10.6 Les métriques

metrics-server collecte toutes les quinze secondes environ la consommation CPU et mémoire auprès des kubelets. Il alimente `kubectl top` et l'autoscaling (module 14). Il ne garde aucun historique.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
# sur kubeadm, les certificats des kubelets sont auto-signés : ajouter --
kubelet-insecure-tls (TP7)
kubectl top nodes
kubectl top pods -A --sort-by=memory
kubectl top pod <pod> --containers
```

L'unité CPU est le millicore (`250m` = un quart de cœur) ; la mémoire s'exprime en Mi ou Gi (mebioctets, gibioctets).

En production, on déploie une chaîne de supervision complète, souvent via le chart Helm `kube-prometheus-stack` :

Brique	Rôle
Prometheus	Collecte et stocke les métriques exposées par les applications et les composants

Brique	Rôle
node-exporter	Métriques système de chaque nœud (DaemonSet)
kube-state-metrics	État des objets Kubernetes (réplicas voulus et disponibles, pods en erreur)
Grafana	Tableaux de bord
Alertmanager	Envoi des alertes (messagerie, astreinte)

Trois questions guident ce qu'on surveille pour une application : combien de requêtes, combien d'erreurs, quel temps de réponse. Pour l'infrastructure : taux d'utilisation, saturation et erreurs de chaque ressource.

10.7 Diagnostiquer le réseau

```
kubectl get endpointslices -l kubernetes.io/service-name=podinfo   # le
Service a-t-il des pods ?
kubectl get pods -l app=podinfo --show-labels                       # les
labels correspondent-ils ?
kubectl run test --image=mirror.gcr.io/library/busybox:1.36 --rm -it --
restart=Never -- sh
# dans le pod de test :
nslookup podinfo                        # le DNS répond-il ?
wget -qO- --timeout=3 http://podinfo:9898   # le Service répond-il ?
nc -zv -w 3 <IP-du-pod> 9898            # le pod répond-il directement ?
```

Si le pod répond en direct mais pas via le Service : `selector` ou `targetPort`. Si rien ne répond depuis un autre namespace : NetworkPolicy (module 15). Si le DNS échoue : état des pods CoreDNS dans `kube-system`.

10.8 Diagnostiquer un nœud

```
kubectl describe node k8s-w1         # conditions, pression, événements
systemctl status kubelet             # [nœud] le kubelet tourne-t-il ?
journalctl -u kubelet -f             # [nœud] ses messages
journalctl -u containerd --since "10 min ago"
sudo crictl ps -a                    # [nœud] conteneurs, y compris arrêtés
df -h /var/lib/containerd            # [nœud] espace disque des images
```

10.9 Les pannes du TP7

Panne	Symptôme	Ce qui la révèle	Correction
Tag d'image inexistant	`ImagePullBackOff`	`describe pod` : image not found	Corriger le tag
Labels du Service et des pods différents	Le Service ne répond pas	EndpointSlice vide	Aligner `selector` et labels
Demande de 8 CPU	`Pending`	`describe pod` : Insufficient cpu	Réduire la demande
Commande erronée	`CrashLoopBackOff`	`describe pod` : executable file not found	Corriger la commande
ConfigMap absente	`CreateContainerConfigError`	`describe pod` : configmap not found	Créer la ConfigMap ou corriger la référence
Readiness sur le mauvais port	`Running` , `0/1`	`describe pod` : connection refused	Corriger le port de la probe

> **TP7 — à faire maintenant** · Jour 2, 14:30 · 60 min
>
> Installer metrics-server, puis diagnostiquer, seul ou en binôme, les six pannes que vous déployez au début du TP avec la méthode de la section 10.1 : les 4 premières pour tous, les 2 dernières en bonus. Le tableau 10.9 sert de corrigé : ne le lisez qu'après.
>
> **Point de contrôle :** `kubectl top nodes` répond, ce qui est indispensable à l'autoscaling du jour 3.
>
> Fin du jour 2 : rattrapage à 15:30, puis quiz à 15:45.

À retenir

- `get` , `describe` (section Events), `logs --previous` , `events` : dans cet ordre.
- Chaque état de pod oriente vers une famille de causes ; `Pending` concerne le placement, `CrashLoopBackOff` l'application.
- Les logs disparaissent avec le pod : centralisez-les en production.

- `kubectl debug` ajoute des outils à un pod qui n'en a pas.
- metrics-server donne l'instant présent ; Prometheus et Grafana donnent l'historique et les alertes.

Questions de révision

1. Un pod est en `CrashLoopBackOff`. Pourquoi `kubectl logs` peut-il ne rien montrer, et quelle option utiliser ?
2. Un pod reste `Pending`. Citez trois causes possibles et la commande qui les distingue.
3. Le Service `podinfo` ne répond pas, mais `curl` sur l'IP d'un pod fonctionne. Où chercher ?
4. Comment ouvrir un shell dans un pod dont l'image ne contient pas de shell ?

Module 11 — Gérer les ressources

Les `requests` réservent des ressources et guident le placement ; les `limits` plafonnent la consommation. Au niveau d'un namespace, une LimitRange fixe des valeurs par défaut et une ResourceQuota un budget total. Bien réglés, ces mécanismes évitent qu'une application mal dimensionnée dégrade tout le cluster.

11.1 Requests et limits

```
containers:
  - name: podinfo
    image: ghcr.io/stefanprodan/podinfo:6.7.1
    resources:
      requests:
        cpu: 100m          # 0,1 cœur réservé
        memory: 128Mi      # 128 mebioctets réservés
      limits:
        cpu: 500m          # au plus un demi-cœur
        memory: 128Mi      # au-delà : arrêt immédiat
```

Unités. CPU : `1` = un vCPU, `500m` = un demi (millicores). Mémoire : suffixes binaires `Mi`, `Gi` (1 Mi = 1 048 576 octets) ou décimaux `M`, `G` (1 M = 1 000 000 octets). Utilisez les suffixes binaires et n'oubliez pas l'unité : `memory: 128` signifie 128 octets.

	`requests`	`limits`
Utilisé par	Le scheduler	Le noyau, via les cgroups

	`requests`	`limits`
Signification	Ce dont le conteneur a besoin pour bien fonctionner	Ce qu'il ne doit jamais dépasser
Effet sur le placement	Un pod ne va que sur un nœud où la somme des requests reste sous l'allouable	Aucun
Effet en fonctionnement	Part de CPU garantie quand le nœud est saturé	CPU : ralentissement. Mémoire : arrêt

Le scheduler raisonne sur les **requests déclarées**, pas sur la consommation réelle : un nœud à 10 % de CPU peut être « plein » si ses pods ont tout réservé.

11.2 Que se passe-t-il en cas de dépassement ?

Ressource	Au-delà de la request	Au-delà de la limit
CPU	Autorisé si le nœud a du CPU libre	Le conteneur est **bridé** (throttling) : il ralentit mais continue
Mémoire	Autorisé, mais le pod devient candidat à l'éviction si le nœud manque de mémoire	Le conteneur est **tué** (`OOMKilled` , code de sortie 137) puis redémarré

Le CPU est une ressource *compressible* : on peut en donner moins. La mémoire ne l'est pas : on ne peut pas reprendre de la mémoire déjà allouée sans arrêter le processus.

11.3 Les classes de qualité de service

Kubernetes range chaque pod dans une classe QoS selon ses déclarations. Elle détermine l'ordre d'éviction quand un nœud manque de mémoire.

Classe	Condition	Priorité en cas de manque
`Guaranteed`	Chaque conteneur a des requests égales à ses limits, pour le CPU et la mémoire	Expulsé en dernier
`Burstable`	Au moins une request ou une limit, sans remplir la condition précédente	Expulsé en second, d'abord ceux qui dépassent le plus leurs requests
`BestEffort`	Aucune request ni limit	Expulsé en premier

```
kubectl get pods -o custom-columns=NOM:.metadata.name,QOS:.status.qosClass
```

11.4 Bien dimensionner

1. **Toujours déclarer des requests.** Sans elles, le scheduler empile les pods à l'aveugle et l'autoscaling ne fonctionne pas.
2. **Mesurer avant de fixer** : observer la consommation réelle (`kubectl top`, Prometheus) sur une période représentative, puis fixer les requests un peu au-dessus de l'usage habituel.
3. **Mémoire : limit égale à la request.** Le pod ne consomme jamais plus que ce qui lui est réservé, et les arrêts pour manque de mémoire deviennent prévisibles.
4. **CPU : limit généreuse, ou absente.** Une limit CPU trop basse ralentit l'application même quand le nœud est inactif. Beaucoup d'équipes ne fixent que la request CPU ; d'autres imposent une limit pour la prévisibilité. Choisissez en connaissance de cause.
5. **Applications Java, Node.js et Python** : vérifiez que leur gestion mémoire tient compte de la limit du conteneur, sinon elles dépassent et sont tuées.

La somme des limits d'un nœud peut dépasser sa capacité : c'est le **surengagement**. Il est acceptable pour le CPU, risqué pour la mémoire.

Les versions récentes de Kubernetes permettent aussi de modifier les requests et limits d'un pod en fonctionnement, sans le recréer (*in-place resize*).

11.5 LimitRange : des valeurs par défaut

Une LimitRange s'applique à un namespace. À la création de chaque pod, elle ajoute les valeurs manquantes et refuse celles qui sortent des bornes. Elle n'agit pas sur les pods déjà créés.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: defauts-equipe-a
  namespace: equipe-a
spec:
  limits:
    - type: Container
      defaultRequest:      # request ajoutée si absente
        cpu: 100m
        memory: 128Mi
      default:             # limit ajoutée si absente
        cpu: 200m
        memory: 128Mi
```

```
      max:                 # au-delà : pod refusé
        cpu: "1"
        memory: 512Mi
      min:
        cpu: 10m
        memory: 16Mi
```

11.6 ResourceQuota : un budget par namespace

Une ResourceQuota plafonne la consommation totale d'un namespace : ressources de calcul, stockage et nombre d'objets.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota-equipe-a
  namespace: equipe-a
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
    pods: "10"
    services: "5"
    persistentvolumeclaims: "4"
    requests.storage: 20Gi
    count/deployments.apps: "5"
```

```
kubectl describe quota -n equipe-a     # consommé / autorisé pour chaque
ligne
```

Deux comportements à connaître :

- Dès qu'un quota porte sur le CPU ou la mémoire, **tout pod du namespace doit déclarer ces ressources**, sinon il est refusé. D'où l'intérêt d'associer une LimitRange qui fournit des valeurs par défaut.
- Un dépassement de quota ne fait pas échouer le Deployment visiblement : le ReplicaSet n'arrive pas à créer ses pods, et l'erreur apparaît dans ses événements (`exceeded quota`).

Le calcul du TP8. Chaque pod reçoit par défaut une request de 128 Mi ; le quota autorise 1 Gi de requests mémoire. 1 Gi / 128 Mi = 8 : le Deployment plafonne à 8 pods sur les 15 demandés, bien avant la limite de 10 pods.

11.7 Un modèle pour plusieurs équipes

Chaque équipe ou environnement reçoit un namespace équipé de cinq objets :

Objet	Rôle	Module
ResourceQuota	Budget total	11
LimitRange	Valeurs par défaut et bornes	11
RoleBinding	Droits de l'équipe, limités à son namespace	15
NetworkPolicy	Isolation réseau par défaut	15
Labels Pod Security	Niveau de sécurité imposé aux pods	15

11.8 Priorités et préemption

Une PriorityClass donne une priorité aux pods. Quand le cluster est plein, le scheduler peut expulser des pods moins prioritaires pour placer un pod plus prioritaire. Les classes `system-cluster-critical` et `system-node-critical` protègent les composants du cluster.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: production-critique
value: 100000
globalDefault: false
description: "Applications de production critiques"
---
# dans le modèle de pod :
# spec:
#   priorityClassName: production-critique
```

À retenir

- Requests = réservation utilisée pour le placement ; limits = plafond appliqué par le noyau.
- Dépasser sa limit CPU ralentit ; dépasser sa limit mémoire tue (`OOMKilled`).
- Les classes QoS fixent l'ordre d'éviction : BestEffort, puis Burstable, puis Guaranteed.
- LimitRange fournit les valeurs par défaut, ResourceQuota le budget ; les deux vont ensemble.
- Mesurez avant de dimensionner ; mémoire : limit = request.

Questions de révision

1. Un nœud affiche 15 % de CPU utilisé mais refuse un nouveau pod pour « Insufficient cpu ». Comment est-ce possible ?
2. Quelle est la classe QoS d'un pod qui déclare `requests.memory: 128Mi` et rien d'autre ?
3. Pourquoi un pod sans ressources est-il refusé dans un namespace qui a un quota CPU ?
4. Quel symptôme observe-t-on quand un conteneur dépasse sa limit mémoire ?

Module 12 — Labels, annotations et stratégies de déploiement

Les labels sont le lien entre les objets : un Service trouve ses pods, un Deployment ses réplicas, une NetworkPolicy ses cibles, tous par sélecteur de labels. Maîtriser ce mécanisme permet de mettre en œuvre, avec les seuls objets natifs, des déploiements blue/green et canary.

12.1 Les labels

Un label est une paire clé-valeur attachée à un objet, destinée à l'**identifier et le sélectionner**.

- **Clé** : un préfixe facultatif en forme de nom de domaine, puis `/`, puis un nom de 63 caractères au plus (lettres, chiffres, `-`, `_`, `.`). Exemple : `app.kubernetes.io/name`. Les préfixes `kubernetes.io/` et `k8s.io/` sont réservés au projet.
- **Valeur** : 63 caractères au plus, mêmes caractères, éventuellement vide. Toujours une chaîne : `"true"`, `"6.7"`.

Le projet recommande un jeu de labels communs, compris par de nombreux outils (Helm, tableaux de bord, Headlamp) :

Label	Signification	Exemple
`app.kubernetes.io/name`	Nom de l'application	`podinfo`
`app.kubernetes.io/instance`	Instance particulière	`podinfo-recette`
`app.kubernetes.io/version`	Version de l'application	`"6.7.1"`
`app.kubernetes.io/component`	Rôle dans l'architecture	`frontend`, `cache`

Label	Signification	Exemple
`app.kubernetes.io/part-of`	Application plus large dont elle fait partie	`fil-rouge`
`app.kubernetes.io/managed-by`	Outil qui la gère	`helm`, `kubectl`

Ajoutez vos propres labels d'organisation : `environnement: recette`, `equipe: diffusion`, `tier: donnees`.

```
kubectl label pods -l app=redis tier=donnees      # ajouter à tous les pods sélectionnés
kubectl label pod <pod> tier=cache --overwrite    # modifier
kubectl label pod <pod> tier-                     # retirer
kubectl get pods --show-labels
kubectl get pods -L app,tier                      # labels en colonnes
```

12.2 Les sélecteurs

Deux familles de sélecteurs :

Type	Opérateurs	Exemple en ligne de commande
Égalité	`=`, `==`, `!=`	`-l app=podinfo,tier!=cache`
Ensemble	`in`, `notin`, existence	`-l 'environnement in (recette,prod),!obsolete'`

Dans les manifestes, les objets récents acceptent les deux formes, les plus anciens seulement l'égalité :

```
# Deployment, ReplicaSet, NetworkPolicy, PodDisruptionBudget, Job
selector:
  matchLabels:
    app: podinfo
  matchExpressions:
    - key: environnement
      operator: In
      values: ["recette", "prod"]

# Service : un simple dictionnaire, égalité uniquement
selector:
  app: podinfo
```

Objet	Sélectionne
Service	Les pods vers lesquels envoyer le trafic
Deployment, ReplicaSet, StatefulSet, DaemonSet	Les pods qu'il gère
NetworkPolicy	Les pods protégés et les sources autorisées
PodDisruptionBudget	Les pods protégés pendant les maintenances
`nodeSelector`, affinités	Les nœuds
`kubectl -l`	Ce que vous affichez ou modifiez

12.3 Les annotations

Une annotation est aussi une paire clé-valeur, mais **non sélectionnable**. Elle documente l'objet ou configure un outil, et sa valeur peut être longue (jusqu'à 256 Kio pour l'ensemble des annotations d'un objet).

Annotation	Posée par	Rôle
`kubernetes.io/change-cause`	Vous	Motif d'une révision dans `rollout history`
`deployment.kubernetes.io/revision`	Le contrôleur Deployment	Numéro de révision
`kubectl.kubernetes.io/last-applied-configuration`	`kubectl apply`	Dernière version appliquée, pour calculer les différences
`kubectl.kubernetes.io/restartedAt`	`kubectl rollout restart`	Force un nouveau rollout
`description`, `contact`	Vous	Documentation libre

```
kubectl annotate deployment podinfo description="Démonstrateur formation
INSEE"
kubectl get deployment podinfo -o jsonpath='{.metadata.annotations}'
```

12.4 Pièges et astuces

- **Le sélecteur d'un Deployment ne se modifie pas** après création. Choisissez-le minimal et stable : `app: podinfo`, jamais un label de version, qui changerait à chaque mise à jour.
- **Deux Deployments aux sélecteurs qui se recouvrent** se disputent les mêmes pods. Chaque sélecteur doit être unique.
- **Mettre un pod en quarantaine** : modifiez un de ses labels pour qu'il sorte du sélecteur. Son ReplicaSet ne le compte plus et crée un remplaçant ; le Service ne lui envoie plus de trafic. Le pod isolé reste disponible pour l'analyse.

```
kubectl label pod podinfo-6c9f8d7b5-4kx2p app=quarantaine --overwrite
```

12.5 Les stratégies de déploiement

Stratégie	Principe	Coupure	Ressources	Retour arrière
Recreate	Tout arrêter, puis tout démarrer	Oui	Aucune en plus	Redéploiement
Rolling update	Remplacer progressivement	Non	Quelques pods en plus	`rollout undo`, progressif
Blue/green	Deux versions complètes, bascule instantanée du trafic	Non	Le double	Instantané
Canary	Une petite part du trafic sur la nouvelle version, puis augmentation	Non	Quelques pods en plus	Instantané

Blue/green avec un Service (démo du jour 3). Deux Deployments complets, distingués par un label `version`. Le Service sélectionne l'un ou l'autre ; la bascule consiste à changer son sélecteur.

```
# Deployment podinfo-blue : pods labellisés app: podinfo-bg, version: blue
(image 6.7.0)
# Deployment podinfo-green : pods labellisés app: podinfo-bg, version: green
(image 6.7.1)
apiVersion: v1
kind: Service
metadata:
  name: podinfo-bg
spec:
  type: NodePort
  selector:
    app: podinfo-bg
    version: blue         # la version qui reçoit le trafic
  ports:
    - port: 9898
      targetPort: 9898
      nodePort: 30081
```

```
kubectl patch service podinfo-bg -p '{"spec":{"selector":{"app":"podinfo-
bg","version":"green"}}}'   # bascule
kubectl patch service podinfo-bg -p '{"spec":{"selector":{"app":"podinfo-
bg","version":"blue"}}}'    # retour
```

Canary avec des réplicas. Le Service sélectionne un label commun aux deux versions ; la répartition suit le nombre de pods.

```
# Service : selector app: podinfo
# Deployment podinfo-stable : 9 réplicas, labels app: podinfo, piste: stable
# Deployment podinfo-canary : 1 réplica,  labels app: podinfo, piste: canary
# => environ 10 % des requêtes vont à la version canary
```

Limite : la proportion dépend du nombre de pods (1 sur 10 au mieux pour 10 pods). Pour un pourcentage précis indépendant des réplicas, la Gateway API pondère le trafic :

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: podinfo
spec:
  parentRefs:
    - name: passerelle-principale
  rules:
    - backendRefs:
        - name: podinfo-stable
          port: 9898
          weight: 95
        - name: podinfo-canary
          port: 9898
          weight: 5
```

Pour automatiser l'augmentation progressive et le retour arrière sur indicateurs (taux d'erreurs, latence), on utilise un outil de *progressive delivery* : Argo Rollouts ou Flagger.

> **TP8 — à faire maintenant** · Jour 3, 09:25 · 45 min
>
> Encadrer un namespace d'équipe avec ResourceQuota et LimitRange, provoquer un `OOMKilled`, puis poser labels et annotations sur le fil rouge (sections 11.1 à 11.6 et 12.1 à 12.3). La bascule blue/green de la section 12.5 est montrée en démo juste avant.
>
> **Point de contrôle :** expliquer pourquoi le déploiement plafonne à 8 pods (calcul de la section 11.6).

À retenir

- Labels pour identifier et sélectionner, annotations pour documenter et configurer des outils.
- Adoptez les labels recommandés `app.kubernetes.io/*`.
- Le sélecteur d'un Deployment est définitif : minimal, stable, sans version.
- Blue/green = changer le sélecteur d'un Service ; canary = partager un label entre deux Deployments, ou pondérer avec la Gateway API.

Questions de révision

1. Pourquoi ne faut-il pas mettre `version: "6.7.0"` dans le sélecteur d'un Deployment ?
2. Que se passe-t-il si vous changez le label `app` d'un pod géré par un ReplicaSet ?
3. Écrivez le sélecteur en ligne de commande pour les pods dont `environnement` vaut `recette` ou `prod` et qui n'ont pas le label `obsolete`.

4. Quel est l'inconvénient principal d'un déploiement blue/green ?

Module 13 — Contrôleurs, opérateurs et Helm

Kubernetes s'étend avec ses propres mécanismes : une CRD ajoute un nouveau type d'objet à l'API, un contrôleur le fait vivre, et l'ensemble forme un opérateur qui automatise l'exploitation d'un logiciel. Helm, lui, empaquette et installe des applications. Les deux se complètent : on installe souvent un opérateur avec Helm.

13.1 Le patron contrôleur

Un contrôleur est un programme qui surveille certains objets via l'API server et agit pour que la réalité corresponde à leur `spec`. Trois propriétés le rendent robuste :

- il réagit à un **état**, pas à un événement : s'il manque une notification, il corrige au passage suivant ;
- ses actions sont **idempotentes** : les rejouer ne crée pas de doublon ;
- il **écrit le résultat** dans le `status` de l'objet, que vous pouvez lire.

Contrôleur intégré	Surveille	Agit sur
Deployment	Deployments	ReplicaSets
ReplicaSet	ReplicaSets, pods	Pods (création, suppression)
Job, CronJob	Jobs, CronJobs	Pods, Jobs
Node	Battements de cœur des nœuds	Conditions, taints, évictions
EndpointSlice	Services, pods	EndpointSlices
Namespace	Namespaces en suppression	Tous les objets du namespace

Propriété et suppression en cascade. Un objet créé par un contrôleur porte une référence à son propriétaire (`ownerReferences`). Supprimer le propriétaire supprime ses dépendants : effacer un Deployment efface ses ReplicaSets et ses pods. L'option `kubectl delete deployment web --cascade=orphan` conserve les dépendants.

Finalizers. Un objet peut porter des *finalizers* : sa suppression est alors différée jusqu'à ce que les contrôleurs concernés aient terminé leur nettoyage (libérer un disque Cloud, par exemple). Un objet bloqué en suppression a souvent un finalizer dont le contrôleur a disparu.

13.2 Étendre l'API avec une CRD

Une **CustomResourceDefinition** (CRD) déclare un nouveau type d'objet. L'API server le stocke, le valide selon son schéma et le sert comme un type natif : `kubectl get`, RBAC, watch.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: sauvegardes.formation.exemple.fr     # <pluriel>.<groupe>
spec:
  group: formation.exemple.fr
  scope: Namespaced
  names:
    kind: Sauvegarde
    plural: sauvegardes
    singular: sauvegarde
    shortNames: ["svg"]
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["base", "planification"]
              properties:
                base:
                  type: string
                planification:
                  type: string
                retention:
                  type: integer
                  minimum: 1
```

```
apiVersion: formation.exemple.fr/v1
kind: Sauvegarde
metadata:
  name: base-diffusion
spec:
  base: diffusion
  planification: "0 3 * * *"
  retention: 7
```

```
kubectl apply -f crd-sauvegarde.yaml
kubectl apply -f sauvegarde-diffusion.yaml
kubectl get sauvegardes          # ou : kubectl get svg
kubectl explain sauvegarde.spec  # la documentation vient du schéma
```

Une CRD seule ne fait que stocker des données : rien ne sauvegarde réellement la base. Il faut un contrôleur qui lise ces objets et agisse.

13.3 Les opérateurs

Un **opérateur** = une ou plusieurs CRD + un contrôleur qui code le savoir-faire d'un administrateur expérimenté de ce logiciel : installer, configurer, mettre à jour, sauvegarder, basculer en cas de panne. Vous déclarez « je veux un cluster PostgreSQL de 3 instances avec une sauvegarde quotidienne » ; l'opérateur s'occupe du reste.

Opérateur	Logiciel géré	Ressources personnalisées
Tigera	Calico (vu au module 4)	`Installation`, `APIServer`
cert-manager	Certificats TLS	`Issuer`, `ClusterIssuer`, `Certificate`
CloudNativePG	PostgreSQL	`Cluster`, `Backup`, `ScheduledBackup`
Prometheus Operator	Supervision	`Prometheus`, `ServiceMonitor`, `PrometheusRule`
Strimzi	Apache Kafka	`Kafka`, `KafkaTopic`
Rook	Stockage Ceph	`CephCluster`, `CephBlockPool`

Le cadre Operator Framework classe les opérateurs en cinq niveaux de maturité : installation simple, mises à jour transparentes, cycle de vie complet (sauvegarde, restauration), supervision approfondie, pilotage automatique (ajustements et réparations autonomes).

Où les trouver : OperatorHub.io et Artifact Hub. Pour en écrire : Kubebuilder ou Operator SDK (Go), kopf (Python).

Avant d'adopter un opérateur, vérifiez sa maturité, sa communauté et les droits qu'il demande : un opérateur tourne souvent avec des permissions étendues sur tout le cluster, et devient un composant de plus à maintenir.

13.4 Exemple : cert-manager (TP9)

cert-manager délivre et renouvelle des certificats TLS. Vous décrivez l'autorité (`ClusterIssuer`) et le certificat voulu (`Certificate`) ; il produit un Secret de type `kubernetes.io/tls` et le renouvelle avant expiration.

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: autosigne
spec:
  selfSigned: {}          # en production : acme (Let's Encrypt) ou
l'autorité interne
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: podinfo-tls
  namespace: fil-rouge
spec:
  secretName: podinfo-tls
  dnsNames:
    - podinfo.fil-rouge.svc.cluster.local
  issuerRef:
    name: autosigne
    kind: ClusterIssuer
```

L'expérience du TP : supprimez le Secret `podinfo-tls`. Quelques secondes plus tard, il est recréé. C'est la réconciliation du module 2, appliquée par un contrôleur qui ne fait pas partie de Kubernetes.

13.5 Helm, le gestionnaire de paquets

Helm installe une application complète (Deployments, Services, ConfigMaps, RBAC…) en une commande, avec des paramètres. Vocabulaire :

Terme	Définition
Chart	Paquet : des modèles de manifestes et leurs valeurs par défaut
Values	Paramètres qui remplissent les modèles (`values.yaml`, `-f`, `--set`)
Release	Une installation d'un chart dans un cluster, avec un nom
Révision	Chaque installation ou mise à jour d'une release, permettant le retour arrière
Dépôt	Serveur qui publie des charts (HTTP ou registre OCI)

Structure d'un chart :

```
mon-app/
├── Chart.yaml          # nom, version du chart, version de l'application
├── values.yaml         # valeurs par défaut
└── templates/
    ├── deployment.yaml # modèles avec des variables {{ .Values.replicaCount
}}
    ├── service.yaml
    └── _helpers.tpl
```

```
helm repo add jetstack https://charts.jetstack.io       # déclarer un dépôt
helm repo update
helm search repo cert-manager
helm show values jetstack/cert-manager > valeurs.yaml   # paramètres
disponibles
helm install cert-manager jetstack/cert-manager -n cert-manager --create-
namespace --set crds.enabled=true
helm upgrade --install cert-manager jetstack/cert-manager -n cert-manager -f
valeurs.yaml   # installer ou mettre à jour
helm list -A                                            # releases installées
helm history cert-manager -n cert-manager
helm rollback cert-manager 1 -n cert-manager
helm template cert-manager jetstack/cert-manager        # afficher les
manifestes sans installer
helm uninstall cert-manager -n cert-manager
```

Helm 4 est sorti fin 2025 ; les commandes ci-dessus sont les mêmes qu'avec Helm 3.

Bonne pratique. Versionnez vos fichiers de values dans Git et fixez la version du chart (`--version`) : `helm upgrade` sans version installe la dernière publiée, peut-être incompatible.

13.6 Helm, Kustomize ou opérateur ?

Outil	Principe	Idéal pour
Kustomize	Superposer des correctifs sur des manifestes simples, sans modèles	Vos applications, déclinées par environnement
Helm	Modèles paramétrables, versionnés et distribués	Installer des logiciels tiers, distribuer une application
Opérateur	Contrôleur qui exploite en continu	Logiciels avec état et opérations complexes (bases de données, files de messages)

Helm installe une fois ; un opérateur surveille et agit en permanence.

> **TP9 — à faire maintenant** · Jour 3, 10:25 · 30 min, guidé
>
> Installer Helm, déployer cert-manager, émettre un certificat et observer sa réconciliation ; prise en main de k9s (sections 13.4 et 13.5).
>
> **Point de contrôle :** le Certificate est `READY True` et son Secret réapparaît après suppression.

À retenir

- Un contrôleur observe, compare, agit, et écrit le résultat dans `status`.
- Une CRD ajoute un type à l'API ; sans contrôleur, elle ne fait que stocker.
- Opérateur = CRD + contrôleur qui automatise l'exploitation d'un logiciel. Vous en utilisez déjà un : Tigera.
- Helm empaquette et installe ; fixez les versions de charts et versionnez vos values.

Questions de révision

1. Pourquoi un objet peut-il rester indéfiniment en état de suppression ?
2. Vous créez une CRD et des objets de ce type, mais rien ne se passe. Pourquoi ?
3. Quelle différence de fonctionnement entre un chart Helm et un opérateur pour gérer une base PostgreSQL ?

4. Quelle commande permet de voir les manifestes qu'un chart va créer, sans rien installer ?

Module 14 — Autoscaling des pods et des nœuds

Kubernetes peut adapter la capacité à la charge sur trois axes : le nombre de pods (HPA), la taille des pods (VPA) et le nombre de nœuds (Cluster Autoscaler, Karpenter). Tous reposent sur des `requests` bien déclarées et sur des métriques fiables.

14.1 Trois axes de mise à l'échelle

Axe	Outil	Ce qui change	Délai typique
Horizontal	HorizontalPodAutoscaler (HPA), intégré	Nombre de réplicas	Secondes à une minute
Vertical	VerticalPodAutoscaler (VPA), à installer	Requests et limits des pods	Au prochain redémarrage du pod, ou en place
Événementiel	KEDA, à installer	Nombre de réplicas, jusqu'à zéro, selon une source externe	Secondes
Cluster	Cluster Autoscaler, Karpenter	Nombre de nœuds	Minutes

14.2 Le HorizontalPodAutoscaler

Toutes les 15 secondes, le contrôleur HPA lit les métriques des pods ciblés et calcule :

```
\text{réplicas voulus} = \left\lceil \text{réplicas actuels} \times
\frac{\text{valeur mesurée}}{\text{valeur cible}} \right\rceil
```

Exemple : 2 réplicas à 90 % de CPU pour une cible de 50 % donnent 2 × 90 / 50 = 3,6, arrondi à 4 réplicas. Un écart de moins de 10 % à la cible ne déclenche rien, pour éviter les oscillations.

Point essentiel : le pourcentage d'utilisation est calculé **par rapport aux requests**. Un pod qui demande 200m et consomme 100m est à 50 %. Sans request, l'HPA affiche `<unknown>` et ne fait rien.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
  behavior:                          # facultatif : vitesse de montée et de
descente
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100                 # au plus doubler...
          periodSeconds: 15          # ...toutes les 15 secondes
    scaleDown:
      stabilizationWindowSeconds: 120  # 300 par défaut
      policies:
        - type: Pods
          value: 1                   # retirer un pod...
          periodSeconds: 60          # ...par minute au plus
```

Comportement par défaut. La montée est rapide. La descente attend 5 minutes de stabilisation : l'HPA retient la plus haute recommandation de cette fenêtre, pour ne pas retirer des pods à la première accalmie.

Types de métriques.

Type	Exemple	Source
`Resource`	CPU ou mémoire moyens des pods	metrics-server
`ContainerResource`	CPU d'un seul conteneur du pod (sans le sidecar)	metrics-server

Type	Exemple	Source
`Pods`	Requêtes par seconde par pod	Métriques personnalisées (adaptateur Prometheus)
`Object`	Longueur d'une file d'attente décrite par un objet	Métriques personnalisées
`External`	Messages en attente dans une file externe	Métriques externes (KEDA, adaptateur)

La mémoire est souvent un mauvais déclencheur : beaucoup d'environnements d'exécution (Java, Go) ne rendent pas la mémoire après un pic, et l'HPA ne redescend jamais.

Piège avec les manifestes. Quand un HPA pilote un Deployment, retirez le champ `replicas` du fichier du Deployment. Sinon, chaque `kubectl apply` remet le nombre de réplicas à la valeur du fichier, à l'encontre de l'HPA.

```
kubectl get hpa -w                          # cible, valeur mesurée, réplicas
kubectl describe hpa php-apache             # conditions AbleToScale,
ScalingActive, ScalingLimited et événements
kubectl autoscale deployment web --min=2 --max=10 --cpu-percent=60   #
création impérative
```

14.3 Le VerticalPodAutoscaler

Le VPA (projet `kubernetes/autoscaler`, à installer) observe la consommation réelle et recommande, ou applique, des requests adaptées.

Mode	Effet
`Off`	Calcule des recommandations sans rien modifier : idéal pour dimensionner
`Initial`	Applique les recommandations seulement à la création des pods
`Recreate`	Recrée les pods dont les requests s'éloignent trop de la recommandation
`InPlaceOrRecreate`	Ajuste les pods en place quand c'est possible, sinon les recrée (versions récentes)

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: podinfo
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  updatePolicy:
    updateMode: "Off"
```

```
kubectl describe vpa podinfo    # section Recommendation : Lower Bound,
Target, Upper Bound
```

Ne combinez pas VPA et HPA sur la même métrique (CPU ou mémoire) : chacun défaisant le travail de l'autre, le système oscille. Le mode `Off` du VPA reste compatible avec tout.

14.4 KEDA : la mise à l'échelle sur événements

KEDA (projet CNCF) déclenche la mise à l'échelle à partir de dizaines de sources : files de messages (Kafka, RabbitMQ), listes Redis, requêtes Prometheus, bases de données, calendrier. Il sait descendre à **zéro réplica** quand il n'y a rien à traiter, et crée lui-même l'HPA correspondant.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: podinfo-heures-ouvrees
spec:
  scaleTargetRef:
    name: podinfo
  minReplicaCount: 0
  maxReplicaCount: 10
  triggers:
    - type: cron
      metadata:
        timezone: Europe/Paris
        start: 0 8 * * 1-5          # 4 réplicas du lundi au vendredi, de 8
h...
        end: 0 19 * * 1-5           # ...à 19 h, zéro le reste du temps
        desiredReplicas: "4"
```

14.5 L'autoscaling des nœuds

Quand l'HPA demande des pods que le cluster ne peut pas placer, ils restent `Pending`. Un autoscaler de cluster réagit alors en ajoutant des nœuds :

- **Cluster Autoscaler** : ajoute un nœud à un groupe de nœuds prédéfini quand des pods sont en attente faute de ressources ; retire les nœuds sous-utilisés après une période d'observation, en déplaçant leurs pods. Il fonctionne avec la plupart des fournisseurs Cloud et avec Cluster API.
- **Karpenter** : choisit et crée directement des machines dimensionnées pour les pods en attente, sans groupes prédéfinis ; il consolide aussi le cluster en remplaçant plusieurs petits nœuds par un plus gros.

Les deux raisonnent sur les **requests** des pods en attente : sans requests, ils ne savent pas quelle capacité ajouter.

Sur nos VM kubeadm, rien ne peut créer de VM à la demande : l'autoscaling des nœuds n'est pas possible. Il faut une couche qui sache provisionner des machines (fournisseur Cloud, ou Cluster API avec un fournisseur OpenStack ou vSphere, par exemple).

14.6 La chaîne complète

1. La charge augmente ; l'utilisation CPU des pods dépasse la cible.
2. L'HPA augmente le nombre de réplicas (quelques secondes).
3. Les nouveaux pods ne trouvent pas de place : ils restent `Pending`.
4. Le Cluster Autoscaler ajoute un nœud (quelques minutes : création de la VM, démarrage, adhésion au cluster).
5. Le scheduler place les pods sur le nouveau nœud ; ils deviennent prêts.

Comme l'ajout d'un nœud prend plusieurs minutes, on garde une marge de capacité. Une technique courante : des pods « réservoirs » de très basse priorité occupent de la place et sont préemptés dès qu'un vrai pod en a besoin (module 11).

14.7 Prérequis d'une application qui passe à l'échelle

- [ ] metrics-server installé et fonctionnel (`kubectl top pods`).
- [ ] Requests CPU et mémoire réalistes sur tous les conteneurs.
- [ ] Application sans état local : n'importe quel réplica peut servir n'importe quelle requête.
- [ ] Readiness probe : les nouveaux pods ne reçoivent du trafic qu'une fois prêts.
- [ ] `minReplicas` d'au moins 2 pour la disponibilité, et un PodDisruptionBudget.
- [ ] Champ `replicas` retiré du manifeste du Deployment.

> **TP10 — à faire maintenant** · Jour 3, 11:15 · 30 min
>
> Créer un HPA à 50 % de CPU sur php-apache, générer de la charge, puis observer la montée et la redescente des réplicas (section 14.2).
>
> **Point de contrôle :** l'HPA dépasse 3 réplicas sous charge.

À retenir

- L'HPA multiplie les pods selon une métrique, calculée par rapport aux requests ; il monte vite et descend après 5 minutes.
- Le VPA recommande ou applique des requests ; en mode `Off`, c'est un outil de dimensionnement.
- KEDA déclenche sur des événements externes et descend à zéro.
- L'autoscaling des nœuds demande une infrastructure pilotable ; il réagit aux pods `Pending`.

Questions de révision

1. 3 réplicas à 80 % de CPU, cible 40 % : combien de réplicas l'HPA demande-t-il ?
2. Pourquoi l'HPA affiche-t-il `<unknown>` pour la métrique CPU ?
3. Pourquoi ne faut-il pas laisser `replicas: 3` dans un Deployment piloté par un HPA ?
4. Pourquoi l'autoscaling des nœuds n'est-il pas possible sur le cluster du TP ?

Module 15 — Sécurité

Par défaut, un cluster Kubernetes est permissif : tout pod peut joindre tout pod, un conteneur peut tourner en root, et un compte trop large ouvre tout. On le sécurise par couches : RBAC limite qui peut faire quoi via l'API, Pod Security limite ce que les pods ont le droit de faire sur le nœud, et les NetworkPolicies limitent qui parle à qui.

15.1 Une sécurité en couches

Le projet décrit quatre couches emboîtées, chacune dépendant de la précédente :

Couche	Exemples de mesures
Cloud ou centre de données	Accès réseau à l'API restreint, nœuds durcis, chiffrement des disques
Cluster	RBAC, Pod Security, NetworkPolicies, chiffrement d'etcd, journaux d'audit, mises à jour

Couche	Exemples de mesures
Conteneur	Images minimales et analysées, utilisateur non root, système de fichiers en lecture seule
Code	Dépendances à jour, secrets hors du code, TLS entre services

15.2 Authentification : qui êtes-vous ?

Kubernetes ne stocke **aucun compte utilisateur**. Il fait confiance à des mécanismes externes :

- **Certificats clients X.509** : le champ CN donne le nom d'utilisateur, le champ O ses groupes. C'est ce qu'utilise `admin.conf`.
- **Jetons OIDC** émis par un fournisseur d'identité (Keycloak, Microsoft Entra ID, Google) : la solution recommandée pour les humains, car elle s'appuie sur l'annuaire de l'organisation.
- **Jetons de ServiceAccount** : l'identité des programmes qui tournent dans le cluster.

Créer un certificat pour une personne, via l'API de signature du cluster :

```
openssl genrsa -out alice.key 2048
openssl req -new -key alice.key -out alice.csr -subj "/CN=alice/O=equipe-
diffusion"
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice
spec:
  request: $(base64 -w0 < alice.csr)
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 2592000      # 30 jours
  usages: ["client auth"]
EOF
kubectl certificate approve alice
kubectl get csr alice -o jsonpath='{.status.certificate}' | base64 -d >
alice.crt
```

15.3 Autorisation RBAC : que pouvez-vous faire ?

RBAC (*Role-Based Access Control*) n'accorde que des permissions : il n'y a pas de règle d'interdiction. Tout ce qui n'est pas explicitement permis est refusé.

Objet	Portée	Contenu
`Role`	Un namespace	Liste de permissions : verbes × ressources
`ClusterRole`	Tout le cluster, ou réutilisable dans chaque namespace	Idem
`RoleBinding`	Un namespace	Accorde un Role ou un ClusterRole à des sujets, dans ce namespace
`ClusterRoleBinding`	Tout le cluster	Accorde un ClusterRole à des sujets, partout

Verbes : `get`, `list`, `watch` (lecture), `create`, `update`, `patch`, `delete`, `deletecollection` (écriture). Sujets : `User`, `Group`, `ServiceAccount`.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: lecture
  namespace: fil-rouge
rules:
  - apiGroups: [""]                 # "" = groupe core (pods, services)
    resources: ["pods", "pods/log", "services"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: equipe-diffusion-lecture
  namespace: fil-rouge
subjects:
  - kind: Group
    name: equipe-diffusion
    apiGroup: rbac.authorization.k8s.io
  - kind: ServiceAccount
    name: lecteur
    namespace: fil-rouge
```

```
roleRef:
  kind: Role
  name: lecture
  apiGroup: rbac.authorization.k8s.io
```

Kubernetes fournit quatre ClusterRoles prêts à l'emploi : `view` (lecture, sans les Secrets), `edit` (lecture et écriture des applications), `admin` (tout dans un namespace, y compris ses droits), `cluster-admin` (tout, partout). Les accorder **par un RoleBinding** les limite à un namespace : c'est le modèle le plus courant pour déléguer un namespace à une équipe.

```
kubectl create rolebinding equipe-a-edit --clusterrole=edit --group=equipe-a
-n equipe-a
kubectl auth can-i delete pods -n fil-rouge --as=system:serviceaccount:fil-
rouge:lecteur
kubectl auth can-i --list -n fil-rouge --as=alice     # tout ce qu'alice peut
faire
```

Droits plus dangereux qu'ils n'en ont l'air :

- `list` ou `watch` sur les Secrets révèle leur contenu ;
- `create` sur les pods permet de monter n'importe quel Secret ou ServiceAccount du namespace, donc d'obtenir leurs droits ;
- les verbes `escalate`, `bind` et `impersonate`, et le joker `*`, ouvrent la porte à l'élévation de privilèges.

15.4 Les ServiceAccounts

Un ServiceAccount est l'identité d'un pod auprès de l'API. Chaque namespace en a un nommé `default`, utilisé si le pod n'en précise pas.

- Le kubelet monte dans chaque pod un **jeton à durée de vie limitée**, renouvelé automatiquement, dans `/var/run/secrets/kubernetes.io/serviceaccount/`.
- Une application qui n'appelle pas l'API n'a pas besoin de ce jeton : `automountServiceAccountToken: false` le supprime et réduit la surface d'attaque.
- Une application qui appelle l'API (un opérateur, un outil de CI) reçoit son propre ServiceAccount et un Role minimal.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: podinfo
  namespace: fil-rouge
automountServiceAccountToken: false
---
# dans le modèle de pod :
# spec:
#   serviceAccountName: podinfo
```

```
kubectl create token lecteur -n fil-rouge --duration=1h   # jeton temporaire, pour un outil ou un test
```

15.5 Pod Security : ce que les pods ont le droit de faire

Les **Pod Security Standards** définissent trois niveaux, appliqués par le contrôleur d'admission **Pod Security Admission**, intégré à Kubernetes :

Niveau	Interdit notamment	Pour
`privileged`	Rien	Composants système (CNI, stockage)
`baseline`	Conteneurs privilégiés, réseau ou PID de l'hôte, volumes `hostPath`, ports de l'hôte, capacités noyau dangereuses	Applications courantes, migration progressive
`restricted`	Tout ce qui précède, plus : exécution en root, élévation de privilèges, capacités non retirées, absence de profil seccomp	Cible pour les applications

Le niveau s'applique par namespace, avec un label par mode :

Mode	Effet d'une violation
`enforce`	Le pod est refusé
`audit`	Le pod est accepté, la violation est notée dans le journal d'audit
`warn`	Le pod est accepté, l'utilisateur voit un avertissement

```
kubectl label namespace equipe-a pod-
security.kubernetes.io/enforce=restricted pod-
security.kubernetes.io/warn=restricted
kubectl label --dry-run=server --overwrite namespace fil-rouge pod-
security.kubernetes.io/enforce=restricted
# la seconde commande liste les pods existants qui seraient en infraction,
sans rien changer
```

Pour des règles plus fines (registres autorisés, labels obligatoires, interdiction de `latest`), on ajoute un moteur de politiques : les **ValidatingAdmissionPolicies** natives (règles en langage CEL), Kyverno ou OPA Gatekeeper.

15.6 Le securityContext

Le `securityContext` déclare comment le conteneur s'exécute. Un pod conforme au niveau `restricted` :

```
apiVersion: v1
kind: Pod
metadata:
  name: podinfo-securise
spec:
  securityContext:                  # niveau pod : s'applique à tous les
conteneurs
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001                  # groupe propriétaire des volumes montés
    seccompProfile:
      type: RuntimeDefault          # filtre les appels système dangereux
  containers:
    - name: podinfo
      image: ghcr.io/stefanprodan/podinfo:6.7.1
      securityContext:              # niveau conteneur
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true   # facultatif pour restricted,
recommandé
        capabilities:
          drop: ["ALL"]
```

Champ	Effet
`runAsNonRoot`	Refuse de démarrer si le processus tournerait en root

Champ	Effet
`runAsUser`, `runAsGroup`	Identifiants numériques imposés au processus
`allowPrivilegeEscalation: false`	Empêche un processus d'acquérir plus de droits que son parent
`capabilities.drop: ["ALL"]`	Retire toutes les capacités noyau (on peut réajouter `NET_BIND_SERVICE`)
`readOnlyRootFilesystem`	Système de fichiers du conteneur en lecture seule ; un volume `emptyDir` pour les écritures temporaires
`privileged: true`	Accès quasi complet à l'hôte : à proscrire hors composants système

15.7 Les NetworkPolicies

Par défaut, tout pod peut joindre tout pod. Dès qu'une NetworkPolicy **sélectionne** un pod pour une direction (entrée ou sortie), seul le trafic qu'une policy autorise explicitement passe dans cette direction. Les policies s'additionnent : il n'y a pas d'ordre ni de règle de refus. Elles n'ont d'effet que si le CNI les applique (Calico : oui).

1. Refus par défaut en entrée pour tout le namespace :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: refus-par-defaut
  namespace: fil-rouge
spec:
  podSelector: {}           # tous les pods du namespace
  policyTypes: ["Ingress"]
```

2. Autoriser podinfo à joindre Redis, et seulement lui :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: podinfo-vers-redis
  namespace: fil-rouge
spec:
  podSelector:
    matchLabels:
      app: redis
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: podinfo
      ports:
        - protocol: TCP
          port: 6379
```

3. Autoriser un autre namespace (par exemple la supervision). Chaque namespace porte automatiquement le label `kubernetes.io/metadata.name` :

```
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: supervision
```

4. Restreindre aussi la sortie : attention, un refus en sortie coupe le DNS. Il faut le réautoriser :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: sortie-dns-seulement
  namespace: fil-rouge
spec:
  podSelector: {}
  policyTypes: ["Egress"]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
```

```
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

Piège du « et » et du « ou ». Dans l'exemple 4, `namespaceSelector` et `podSelector` sont dans **le même élément** de liste (un seul tiret) : la source doit satisfaire les deux (pods `kube-dns` **du** namespace `kube-system`). Avec deux tirets, ce serait deux sources distinctes : tous les pods de `kube-system` **ou** les pods `kube-dns` du namespace de la policy, ici `fil-rouge`, où il n'y en a aucun. Un `podSelector` seul ne regarde en effet que son propre namespace.

```
kubectl get networkpolicies -n fil-rouge
kubectl run intrus -n fil-rouge --image=mirror.gcr.io/library/busybox:1.36 --
rm -it --restart=Never -- nc -zv -w 3 redis 6379
```

15.8 Les autres mesures

- **Mettre à jour** le cluster régulièrement : chaque version mineure n'est maintenue qu'environ 14 mois.
- **Restreindre l'accès réseau** à l'API server (port 6443) aux postes d'administration et à la chaîne de déploiement.
- **Chiffrer les Secrets** dans etcd (module 8) et protéger les sauvegardes d'etcd.
- **Activer les journaux d'audit** de l'API server pour savoir qui a fait quoi.
- **Analyser les images** (Trivy) et vérifier leur provenance (signatures Sigstore et cosign).
- **Détecter les comportements anormaux** à l'exécution (Falco).
- **Contrôler la configuration** du cluster avec le référentiel CIS Kubernetes Benchmark (outil kube-bench).

> **TP11 — à faire maintenant** · Jour 3, 13:20 · 50 min
>
> Créer un compte en lecture seule, imposer Pod Security `restricted`, isoler Redis par NetworkPolicies, puis créer l'utilisatrice alice par certificat (sections 15.2 à 15.7).
>
> **Point de contrôle :** le compte `lecteur` ne peut rien supprimer ; un pod intrus ne joint pas Redis alors que l'application fonctionne.

À retenir

- Kubernetes n'a pas de comptes utilisateurs : certificats ou OIDC pour les humains, ServiceAccounts pour les programmes.
- RBAC n'accorde que des permissions ; déléguez un namespace avec un RoleBinding vers `edit` ou `admin`.
- Lire les Secrets ou créer des pods sont des droits sensibles.
- Pod Security Admission impose `baseline` ou `restricted` par label de namespace ; visez `restricted`.
- Une NetworkPolicy transforme le « tout est permis » en liste blanche pour les pods qu'elle sélectionne ; n'oubliez pas le DNS en sortie.

Questions de révision

1. Comment donner à une équipe le droit de tout gérer dans son namespace, et rien ailleurs ?
2. Pourquoi le droit de créer des pods permet-il de lire les Secrets du namespace ?
3. Un pod est refusé avec le message `violates PodSecurity "restricted:latest"`. Que faut-il ajouter à son manifeste ?
4. Après une policy de refus en sortie, plus aucun nom de service ne se résout. Pourquoi ?
5. Quelle différence entre un `namespaceSelector` et un `podSelector` placés dans le même élément `from`, et dans deux éléments séparés ?

Module 16 — Volumes et données persistantes

Le système de fichiers d'un conteneur disparaît avec lui ; pour que des données survivent, le pod demande un volume persistant par un PersistentVolumeClaim, que Kubernetes associe à un PersistentVolume fourni à la main ou créé à la demande par une StorageClass. Pour les bases de données, le StatefulSet ajoute une identité et un volume propres à chaque réplica.

16.1 Le problème

Trois raisons de perdre des données écrites dans un conteneur :

1. le conteneur redémarre (plantage, liveness) : sa couche inscriptible est remise à zéro ;
2. le pod est supprimé ou remplacé (mise à jour, éviction) ;
3. le pod est recréé sur un autre nœud.

Au TP12, vous le constatez : une clé écrite dans Redis disparaît dès que le pod Redis est remplacé.

16.2 Les types de volumes

Un volume est déclaré au niveau du pod (`volumes`) et monté dans un ou plusieurs conteneurs (`volumeMounts`).

Type	Durée de vie	Usage
`emptyDir`	Celle du pod (survit aux redémarrages du conteneur)	Fichiers temporaires, cache, échange entre conteneurs d'un pod
`configMap`, `secret`, `downwardAPI`, `projected`	Celle de l'objet source	Configuration en fichiers (module 8)
`persistentVolumeClaim`	Indépendante du pod	Données persistantes
Volume éphémère générique (`ephemeral`)	Celle du pod, mais sur un vrai stockage	Grand espace de travail temporaire
`hostPath`	Celle du nœud	Composants système seulement ; interdit par Pod Security `baseline`

```
spec:
  containers:
    - name: app
      image: mirror.gcr.io/library/busybox:1.36
      command: ["sh", "-c", "date > /travail/debut.txt; sleep 3600"]
      volumeMounts:
        - name: travail
          mountPath: /travail
  volumes:
    - name: travail
      emptyDir:
        sizeLimit: 500Mi          # medium: Memory pour un disque en mémoire
```

16.3 PersistentVolume, PersistentVolumeClaim, StorageClass

Objet	Portée	Qui le crée	Analogie
PersistentVolume (PV)	Cluster	L'administrateur, ou le provisionneur automatiquement	Un disque disponible

Objet	Portée	Qui le crée	Analogie
PersistentVolumeClaim (PVC)	Namespace	Le développeur	Un bon de commande : « 10 Gi, lecture-écriture »
StorageClass	Cluster	L'administrateur	Le catalogue : quel type de disque, chez quel fournisseur

Deux modes de provisionnement :

- **Statique** : l'administrateur crée des PV à l'avance ; Kubernetes associe chaque PVC à un PV compatible (taille, mode d'accès, classe). C'est ce que vous faites au TP12.
- **Dynamique** : le PVC nomme une StorageClass ; son provisionneur crée un disque et le PV correspondant à la volée. C'est le fonctionnement normal en production.

Un PV et un PVC sont liés un pour un. Le pod ne référence que le PVC : il ignore tout du stockage réel, ce qui rend le manifeste portable d'un cluster à l'autre.

Modes d'accès.

Mode	Abréviation	Signification
`ReadWriteOnce`	RWO	Lecture-écriture par **un seul nœud** (plusieurs pods de ce nœud peuvent l'utiliser)
`ReadWriteOncePod`	RWOP	Lecture-écriture par un seul pod dans tout le cluster
`ReadOnlyMany`	ROX	Lecture seule par plusieurs nœuds
`ReadWriteMany`	RWX	Lecture-écriture par plusieurs nœuds : stockage de fichiers partagé (NFS, CephFS)

Un disque bloc (disque Cloud, Ceph RBD) n'offre que RWO ou RWOP ; il faut un système de fichiers réseau pour RWX.

Autres paramètres.

- `persistentVolumeReclaimPolicy` : `Delete` (le disque est détruit avec le PVC, défaut du provisionnement dynamique) ou `Retain` (le PV passe `Released` et garde ses données jusqu'à intervention).
- `volumeBindingMode` : `Immediate` (le volume est choisi dès la création du PVC) ou `WaitForFirstConsumer` (attend le placement du pod, pour choisir un volume accessible depuis son nœud ou sa zone).

- `allowVolumeExpansion: true` : un PVC peut ensuite être agrandi en modifiant sa taille demandée.
- États d'un PV : `Available` (libre), `Bound` (lié), `Released` (PVC supprimé, données conservées), `Failed`.

16.4 Les pilotes CSI

Le stockage passe par des pilotes **CSI** (Container Storage Interface), fournis par chaque solution : disques des Clouds publics, Cinder pour OpenStack, vSphere, Ceph via Rook, NFS (`csi-driver-nfs`), baies NetApp, Longhorn. Exemple de StorageClass NFS, marquée comme classe par défaut :

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-partage
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: nfs.csi.k8s.io
parameters:
  server: nfs.exemple.fr
  share: /exports/kubernetes
reclaimPolicy: Delete
volumeBindingMode: Immediate
allowVolumeExpansion: true
```

Un PVC qui ne précise pas de classe reçoit la classe par défaut. Beaucoup de pilotes CSI savent aussi prendre des instantanés (objets `VolumeSnapshot`).

```
kubectl get storageclass
kubectl get pv,pvc -A
kubectl describe pvc redis-data -n fil-rouge   # événements de liaison et de
provisionnement
```

16.5 Volume local : le TP12

Sans stockage réseau, le TP utilise un **volume local** : un répertoire de `k8s-w1`, déclaré à la main. Le PV porte une affinité de nœud : tout pod qui l'utilise sera placé sur `k8s-w1`.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-storage
provisioner: kubernetes.io/no-provisioner   # pas de création automatique
```

```
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: redis-pv
spec:
  capacity:
    storage: 1Gi
  accessModes: ["ReadWriteOnce"]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-storage
  local:
    path: /mnt/data/redis
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values: ["k8s-w1"]
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: redis-data
  namespace: fil-rouge
spec:
  storageClassName: local-storage
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
```

Le Deployment Redis monte ce PVC sur `/data`, active la persistance de Redis (`--appendonly yes`) et passe en stratégie `Recreate` : deux instances Redis ne doivent jamais écrire les mêmes fichiers en même temps.

Limite : si `k8s-w1` tombe, Redis ne peut redémarrer nulle part. Un volume local convient aux tests, ou aux bases qui répliquent elles-mêmes leurs données entre nœuds ; en production, on préfère un stockage réseau.

16.6 Le StatefulSet

Un Deployment traite ses pods comme interchangeables. Une base de données a besoin de plus :

Garantie	Deployment	StatefulSet
Nom des pods	Aléatoire (`redis-6c9f8-x2x7k`)	Stable et ordonné (`redis-0` , `redis-1`)
Nom DNS par pod	Non	Oui, via un Service headless : `redis-0.redis.fil-rouge.svc.cluster.local`
Volume	Partagé par tous les pods	Un PVC par pod, créé depuis `volumeClaimTemplates` , qui suit le pod
Ordre de démarrage et d'arrêt	Aucun	`-0` , puis `-1` … ; arrêt en ordre inverse

```
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  clusterIP: None               # Service headless, requis par le
StatefulSet
  selector:
    app: redis
  ports:
    - port: 6379
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: mirror.gcr.io/library/redis:7-alpine
          args: ["--appendonly", "yes"]
```

```
          volumeMounts:
            - name: data
              mountPath: /data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

Par défaut, les PVC créés par un StatefulSet sont conservés quand on le réduit ou le supprime : les données ne disparaissent pas par erreur. Un StatefulSet ne réplique pas les données : c'est à la base de le faire, d'où l'intérêt des opérateurs comme CloudNativePG (module 13).

16.7 Sauvegarder

- **Instantanés de volumes** (`VolumeSnapshot`) : rapides, mais liés au système de stockage.
- **Velero** : sauvegarde les objets Kubernetes et le contenu des volumes vers un stockage objet, et restaure un namespace ou un cluster entier.
- **Sauvegardes natives** des bases de données (dump, archivage des journaux), souvent pilotées par leur opérateur.
- Une sauvegarde n'existe vraiment que si la restauration a été testée.

> **TP12 — à faire maintenant** · Jour 3, 14:15 · 45 min
>
> Constater la perte des données de Redis, puis les rendre persistantes avec une StorageClass locale, un PV et un PVC (sections 16.3 et 16.5).
>
> **Point de contrôle :** la clé écrite dans Redis survit à la suppression de son pod.

À retenir

- Le pod demande (PVC), l'administrateur ou la StorageClass fournit (PV) ; le manifeste reste portable.
- RWO = un seul nœud, pas un seul pod ; RWX exige un stockage de fichiers partagé.
- `WaitForFirstConsumer` choisit le volume après le placement du pod.
- StatefulSet = noms stables, DNS par pod, un volume par réplica ; la réplication reste l'affaire de l'application.
- Pas de données persistantes sans plan de sauvegarde et de restauration testé.

Questions de révision

1. Un PVC reste `Pending`. Citez deux causes possibles avec le provisionnement statique.
2. Pourquoi le pod Redis du TP12 est-il toujours placé sur `k8s-w1` ?
3. Deux pods sur deux nœuds différents doivent écrire dans le même volume. Quel mode d'accès, et quel type de stockage ?
4. Qu'apporte un StatefulSet par rapport à un Deployment pour une base de données ?

Atelier de synthèse — Migrer une application de VM vers Kubernetes

Migrer une application revient à répondre à onze questions, dont chaque réponse désigne un objet Kubernetes vu pendant la formation. L'atelier du jour 3 (TP13) applique cette grille à vos propres projets : le cluster Elastic de l'équipe Observabilité, un service de l'équipe Secrets, une application de l'assistance utilisateurs. Chaque groupe remplit la grille, puis présente en 3 minutes les objets retenus et le point le plus délicat.

> **TP13 — à faire maintenant** · Jour 3, 15:00 · 30 min
>
> Par équipe, remplir la grille ci-dessous pour votre propre application (15 min), puis la présenter en 3 minutes. La formation se termine à 15:30 par le QCM final, le bilan et l'évaluation à chaud.

La grille

Question	Ce qu'il faut regarder sur la VM	Réponse Kubernetes	Module
L'application garde-t-elle un état local ?	Fichiers écrits, sessions en mémoire, base embarquée	Sans état : Deployment ; avec état : StatefulSet ou opérateur	7, 13, 16
Où vont ses données, en quelle quantité ?	Volumes, taille, performances disque	PVC, StorageClass, mode d'accès, sauvegarde	16
Comment est-elle configurée ?	Fichiers de configuration, variables	ConfigMaps, fichiers montés	8
Quels secrets utilise-t-elle ?	Mots de passe, certificats, jetons	Secrets, coffre externe, RBAC	8, 15

Question	Ce qu'il faut regarder sur la VM	Réponse Kubernetes	Module
Comment sait-on qu'elle fonctionne ?	Supervision actuelle, URL de santé	startupProbe, readinessProbe, livenessProbe	9
De quelles ressources a-t-elle besoin ?	CPU et mémoire réellement consommés	requests, limits, classe QoS	11
Qui l'appelle, qui appelle-t-elle ?	Flux réseau, ports, règles de pare-feu	Services, DNS, entrée HTTP, NetworkPolicies	7, 15
Comment survit-elle à une panne, comment passe-t-elle à l'échelle ?	Redondance actuelle, pics de charge	Réplicas, anti-affinité, PodDisruptionBudget, HPA	5, 14
Comment l'observe-t-on ?	Fichiers de logs, métriques	Logs sur la sortie standard, métriques Prometheus	10
Quels droits lui faut-il sur la machine ?	Exécution en root, ports bas, accès système	securityContext, Pod Security	15
Comment est-elle livrée ?	Paquets, scripts d'installation	Image versionnée, Helm ou Kustomize, GitOps	6, 13

Exemple : un cluster Elasticsearch

Elasticsearch a un état (ses index), des besoins mémoire élevés et une topologie à respecter : c'est le cas typique où un opérateur fait mieux qu'un StatefulSet écrit à la main. Elastic publie le sien, ECK (Elastic Cloud on Kubernetes), qui s'installe avec Helm (exemple d'installation) :

```
helm repo add elastic https://helm.elastic.co
helm repo update
helm install elastic-operator elastic/eck-operator -n elastic-system --
create-namespace
```

On décrit ensuite le cluster voulu dans une ressource personnalisée. L'opérateur crée les StatefulSets, les Services, les Secrets, les certificats et un PodDisruptionBudget, puis gère les mises à jour nœud par nœud.

```
apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
  name: logs
  namespace: observabilite
spec:
  version: <version d'Elasticsearch cible>
  nodeSets:
    - name: donnees
      count: 3
      podTemplate:
        spec:
          containers:
            - name: elasticsearch
              resources:
                requests:
                  cpu: "1"
                  memory: 4Gi
                limits:
                  cpu: "1"
                  memory: 4Gi           # Elasticsearch dimensionne son heap
à partir de cette limite
      volumeClaimTemplates:
        - metadata:
            name: elasticsearch-data     # nom attendu par ECK pour le
dossier des données
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: <classe de stockage rapide>
            resources:
              requests:
                storage: 100Gi
```

Points d'attention pour cette migration :

- **Paramètre noyau** : en production, Elasticsearch exige `vm.max_map_count` à 262144 au moins sur chaque nœud. On le règle sur les nœuds, ou par un conteneur d'initialisation privilégié ; `node.store.allow_mmap: false` le contourne pour des tests.
- **Mémoire** : la limite mémoire du conteneur fixe la taille du heap Java. Des requests égales aux limits, pour le CPU comme pour la mémoire, donnent la classe QoS Guaranteed.
- **Stockage** : un volume par nœud Elasticsearch, sur une classe de stockage performante. Prévoyez la capacité et les snapshots Elasticsearch vers un stockage objet.

- **Répartition** : les nœuds Elasticsearch doivent tourner sur des nœuds Kubernetes différents ; ECK pose une anti-affinité préférée par défaut, à rendre obligatoire si besoin.
- **Bascule** : migrez par snapshot et restauration ou par réindexation, pendant que l'ancien cluster sur VM continue de servir.

Annexe A — Mémo kubectl

Les commandes les plus utiles, regroupées par usage. `<ns>` désigne un namespace, `<p>` un pod.

```
# --- Contexte et découverte ---
kubectl config get-contexts
kubectl config use-context <contexte>
kubectl config set-context --current --namespace=<ns>
kubectl cluster-info
kubectl version
kubectl api-resources                        # types, noms courts, portée
kubectl explain deployment.spec.strategy     # documentation d'un champ

# --- Lire ---
kubectl get pods -o wide                     # IP et nœud
kubectl get all -n <ns>                      # principaux objets d'un
namespace
kubectl get pods -A -l app=podinfo --show-labels
kubectl get pod <p> -o yaml
kubectl get pods -o jsonpath='{.items[*].spec.nodeName}'
kubectl get pods -o custom-columns=NOM:.metadata.name,QOS:.status.qosClass
kubectl describe <type> <nom>                # détail + Events
kubectl events --types=Warning
kubectl top nodes ; kubectl top pods -A --sort-by=memory

# --- Créer et modifier ---
kubectl apply -f fichier.yaml                # ou -f dossier/ , ou -k
overlay/
kubectl diff -f fichier.yaml
kubectl apply -f fichier.yaml --dry-run=server
kubectl delete -f fichier.yaml
kubectl create deployment web --image=<image> --dry-run=client -o yaml >
web.yaml
kubectl edit deployment web
kubectl label pod <p> cle=valeur [--overwrite] ; kubectl label pod <p> cle-
kubectl annotate deployment web description="..."
kubectl patch service web -p '{"spec":{"selector":{"version":"green"}}}'
```

```
# --- Déploiements ---
kubectl scale deployment web --replicas=5
kubectl set image deployment/web web=<image>:<tag>
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout undo deployment/web [--to-revision=2]
kubectl rollout restart deployment/web

# --- Diagnostic ---
kubectl logs <p> [-c <conteneur>] [-f] [--previous] [--since=10m]
kubectl logs -l app=web --prefix
kubectl exec -it <p> -- sh
kubectl port-forward svc/web 8080:80
kubectl debug -it <p> --image=mirror.gcr.io/library/busybox:1.36 --target=
<conteneur>
kubectl run tmp --image=mirror.gcr.io/library/busybox:1.36 --rm -it --
restart=Never -- sh

# --- Nœuds ---
kubectl cordon <nœud> ; kubectl uncordon <nœud>
kubectl drain <nœud> --ignore-daemonsets --delete-emptydir-data
kubectl taint nodes <nœud> cle=valeur:NoSchedule      # ajouter un tiret
final pour retirer

# --- Sécurité ---
kubectl auth can-i delete pods -n <ns> --as=system:serviceaccount:<ns>:<sa>
kubectl auth can-i --list -n <ns>
kubectl create token <sa> -n <ns> --duration=1h

# --- Côté nœud (sans passer par l'API) ---
sudo crictl ps -a ; sudo crictl logs <id>
journalctl -u kubelet -f
sudo kubeadm certs check-expiration
```

Annexe B — Glossaire

Terme	Définition
Admission (contrôleur d')	Code de l'API server qui modifie ou refuse un objet avant son enregistrement (LimitRange, Pod Security)
Annotation	Métadonnée clé-valeur non sélectionnable, pour documenter ou configurer un outil

Terme	Définition
API server	Point d'entrée unique du cluster, seul composant à parler à etcd
CNI	Interface et plugins qui fournissent le réseau des pods (Calico, Cilium)
ConfigMap	Objet qui stocke une configuration non sensible
Contrôleur	Boucle qui ramène l'état réel vers l'état désiré
CoreDNS	Serveur DNS interne du cluster
CRD	Définition d'un nouveau type d'objet dans l'API
CRI	Interface entre le kubelet et le runtime de conteneurs
CronJob	Contrôleur qui crée des Jobs selon un calendrier
CSI	Interface entre Kubernetes et les solutions de stockage
DaemonSet	Contrôleur qui fait tourner un pod par nœud
Deployment	Contrôleur d'application sans état : réplicas, mises à jour, retours arrière
Drain	Vider un nœud de ses pods avant maintenance
EndpointSlice	Liste des adresses des pods derrière un Service
etcd	Base clé-valeur qui contient tout l'état du cluster
HPA	HorizontalPodAutoscaler : ajuste le nombre de réplicas selon une métrique
Image	Paquet en lecture seule (couches et configuration) dont on lance des conteneurs
Job	Contrôleur qui exécute des pods jusqu'à leur terme
kubeadm	Outil officiel d'installation et de mise à jour d'un cluster
kubeconfig	Fichier qui décrit clusters, identités et contextes pour kubectl
kubectl	Client en ligne de commande de l'API
kubelet	Agent de chaque nœud, qui fait tourner les pods

Terme	Définition
kube-proxy	Composant qui programme les règles réseau des Services sur chaque nœud
Label	Métadonnée clé-valeur qui identifie et permet de sélectionner un objet
LimitRange	Valeurs par défaut et bornes de ressources dans un namespace
Namespace	Espace de noms qui regroupe des objets et porte quotas et droits
NetworkPolicy	Règle de flux réseau autorisés vers ou depuis des pods
Nœud	Machine (physique ou virtuelle) membre du cluster
Opérateur	CRD + contrôleur qui automatise l'exploitation d'un logiciel
Pod	Plus petite unité déployable : un ou plusieurs conteneurs partageant réseau et volumes
Pod statique	Pod lancé directement par le kubelet depuis un fichier local
Probe	Test de santé régulier d'un conteneur (startup, readiness, liveness)
PV	PersistentVolume : un volume de stockage disponible dans le cluster
PVC	PersistentVolumeClaim : demande de stockage faite par une application
QoS	Classe de qualité de service d'un pod (Guaranteed, Burstable, BestEffort)
RBAC	Contrôle d'accès par rôles : Roles et RoleBindings
ReplicaSet	Contrôleur qui maintient un nombre de pods identiques
Request, limit	Ressources réservées et plafond d'un conteneur
ResourceQuota	Budget de ressources et d'objets d'un namespace
Scheduler	Composant qui choisit le nœud de chaque pod

Terme	Définition
Secret	Objet qui stocke une donnée sensible, encodée en base64
Selector	Critère de sélection d'objets par leurs labels
Service	Adresse, nom DNS et répartition de charge stables devant des pods
ServiceAccount	Identité d'un pod auprès de l'API
StatefulSet	Contrôleur pour applications avec état : noms stables, un volume par réplica
StorageClass	Modèle de stockage utilisé pour créer des volumes à la demande
Taint, tolérance	Marque qui repousse les pods d'un nœud ; autorisation d'un pod à l'ignorer

Annexe C — Corrigé des questions de révision

Module 1

1. Les cgroups, qui plafonnent la mémoire d'un groupe de processus.
2. L'image `redis`, tag `7-alpine`, sur Docker Hub : `docker.io/library/redis:7-alpine`.
3. Les images respectent le format OCI ; containerd les exécute pour le kubelet via le CRI.
4. Placement sur plusieurs hôtes, relance après panne d'un serveur, mise à l'échelle, mises à jour sans coupure, découverte de services.

Module 2

1. Il doit tourner avant tout pod, y compris ceux du control plane qu'il lance lui-même comme pods statiques.
2. Le scheduler : aucun nœud ne passe le filtrage (ressources, taint, affinité, volume).
3. Le contrôleur ReplicaSet constate l'écart et crée un pod, le scheduler le place, le kubelet le démarre, l'EndpointSlice du Service est mis à jour.
4. Elles continuent de tourner, mais plus aucune modification, placement ni auto-réparation.

Module 3

1. Au choix : applications, RBAC, quotas, NetworkPolicies, sécurité des images.
2. Rien ne fournit d'adresse externe : il faut un contrôleur de Cloud ou une solution comme MetalLB.

3. Développement, tests de manifestes, intégration continue, apprentissage rapide.

Module 4

1. Aucun plugin CNI n'est installé : il n'y a pas encore de réseau des pods.
2. Le nœud vérifie qu'il rejoint le bon cluster en comparant l'empreinte de la CA, ce qui empêche l'usurpation.
3. En éditant `/etc/kubernetes/manifests/kube-apiserver.yaml` ; le kubelet redémarre alors le pod.
4. Pour qu'une mise à jour système ne change jamais la version de Kubernetes hors procédure.
5. Les réseaux Cloud rejettent souvent les paquets aux adresses sources inconnues ; VXLAN encapsule le trafic des pods entre adresses de nœuds.

Module 5

1. `cordon` empêche les nouveaux placements ; `drain` fait un cordon puis évince les pods.
2. Une tolérance autorise sans attirer : il faut y ajouter une affinité vers le nœud.
3. Une anti-affinité de pods ou une contrainte de répartition sur `kubernetes.io/hostname`.
4. Ce sont des DaemonSets qui tolèrent la taint du control plane.

Module 6

1. `1.10` est lu comme le nombre 1,1 ; un label doit être une chaîne : `"1.10"`.
2. `create` échoue (l'objet existe déjà) ; `apply` le met à jour.
3. `kubectl create cronjob test --image=<image> --schedule="*/5 * * * *" --dry-run=client -o yaml`.
4. Le prochain `apply` remet l'image du fichier ; il faut reporter la correction dans le fichier.

Module 7

1. Non : ils partagent la même pile réseau, un port ne peut être ouvert qu'une fois.
2. Le Service n'a aucun endpoint : les connexions échouent.
3. `redis.fil-rouge`, ou le nom complet `redis.fil-rouge.svc.cluster.local`.
4. S'il meurt ou si son nœud tombe, rien ne le recrée.
5. NodePort ouvre un port sur chaque nœud ; LoadBalancer demande en plus un répartiteur externe au fournisseur.

Module 8

1. Ils ne voient rien : les variables sont lues au démarrage. Il faut `kubectl rollout restart`.
2. `list` renvoie les objets complets, valeurs comprises.
3. `data` attend des valeurs en base64 ; `stringData` du texte clair, encodé par l'API.
4. Sealed Secrets, SOPS, ou un coffre-fort externe synchronisé par External Secrets Operator.

Module 9

1. Au plus 13 pods (10 + 25 % arrondi à 3), au moins 8 disponibles (25 % arrondi à 2).
2. Grâce à `maxUnavailable`, d'anciens pods prêts restent en place ; les nouveaux, jamais prêts, ne reçoivent pas de trafic.
3. Tous les pods redémarrent en boucle et l'incident s'aggrave.
4. `kubectl create job test --from=cronjob/<nom>`.

Module 10

1. Le conteneur courant vient de redémarrer et n'a encore rien écrit : utilisez `--previous`.
2. Ressources insuffisantes, taint non tolérée, PVC non lié, nodeSelector impossible : `kubectl describe pod` affiche la raison (FailedScheduling).
3. Dans le `selector` du Service, les labels des pods et le `targetPort` ; vérifier l'EndpointSlice.
4. Avec `kubectl debug`, qui ajoute un conteneur éphémère équipé.

Module 11

1. Le scheduler compte les requests réservées, pas la consommation réelle.
2. Burstable.
3. Avec un quota CPU, tout pod doit déclarer ses ressources CPU ; sans LimitRange pour les fournir, il est refusé.
4. Le conteneur est tué (`OOMKilled`, code 137) puis redémarré ; RESTARTS augmente.

Module 12

1. Le sélecteur est définitif alors que la version change à chaque mise à jour.
2. Le pod sort du ReplicaSet, qui crée un remplaçant ; le pod isolé reste, sans trafic.
3. `-l 'environnement in (recette,prod),!obsolete'`.
4. Il faut le double de ressources pendant la bascule.

Module 13

1. Un finalizer attend un contrôleur qui ne répond plus ou a été désinstallé.

2. Aucun contrôleur ne gère ce type : la CRD ne fait que stocker.
3. Le chart installe une fois ; l'opérateur surveille et agit en continu (bascule, sauvegardes, mises à jour).
4. `helm template`.

Module 14

1. 3 × 80 / 40 = 6 réplicas.
2. Les pods n'ont pas de request CPU, ou metrics-server ne fonctionne pas.
3. Chaque `kubectl apply` remet 3 réplicas, à l'encontre de l'HPA.
4. Rien ne peut créer de VM à la demande dans cet environnement.

Module 15

1. Un RoleBinding dans son namespace, vers le ClusterRole `admin` (ou `edit`), pour le groupe de l'équipe.
2. Un pod peut monter n'importe quel Secret de son namespace et en afficher le contenu.
3. Un `securityContext` avec `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, `capabilities.drop: ["ALL"]` et `seccompProfile.type: RuntimeDefault`.
4. Le refus en sortie bloque les requêtes DNS vers CoreDNS : il faut autoriser UDP et TCP 53 vers les pods `kube-dns`.
5. Même élément : « et » (ces pods, dans ces namespaces). Éléments séparés : « ou » ; un `podSelector` seul ne vise alors que le namespace de la policy.

Module 16

1. Aucun PV compatible (taille, mode d'accès, classe) ; ou, avec `WaitForFirstConsumer`, aucun pod ne l'utilise encore, ce qui est normal.
2. Le PV local porte une affinité de nœud vers `k8s-w1`.
3. `ReadWriteMany`, sur un stockage de fichiers partagé (NFS, CephFS).
4. Des noms et un DNS stables par pod, un volume par réplica et un démarrage ordonné.

Annexe D — Pour aller plus loin

- Documentation officielle de Kubernetes : concepts, tâches, référence de l'API ; une partie est traduite en français.
- Kubernetes the Hard Way : monter un cluster sans kubeadm, composant par composant.
- Killercoda : laboratoires Kubernetes gratuits dans le navigateur, dont des scénarios de préparation aux certifications.
- CNCF Landscape : carte des projets de l'écosystème.

- Artifact Hub et OperatorHub : catalogues de charts Helm et d'opérateurs.
- Headlamp : l'interface web présentée en démonstration au jour 3.
- Onyxia : la plateforme de datascience de l'INSEE, bâtie sur Kubernetes.

Sources

Versions et changements récents cités dans ce support :

- Kubernetes v1.34 : détection automatique du pilote de cgroups et fin du support de containerd 1.x (blog Kubernetes)
- Kubernetes 1.37 : sortie le 26 août 2026 et changements à anticiper
- Archivage du Kubernetes Dashboard (21 janvier 2026)
- Installation de Calico 3.31 avec l'opérateur Tigera